

MASCOPE: DIAGNOSING COLLABORATION LOSS IN MULTI-AGENT SYSTEMS

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-agent systems assemble agents with different expertise, yet performance depends on how that expertise is organized, not on how much of it is present. Existing benchmarks evaluate the final outcome and cannot tell what collaboration contributed to it. We introduce MAScope, an environment of 51 expert agents with private knowledge from Stack Exchange and 2,766 collaborative tasks with verifiable evidence and work dependencies. MAScope follows each required contribution from the agent that produces it into the final answer, splitting collaboration into four measurable capabilities: *expert selection*, *task orchestration*, *information transfer*, and *result integration*. Experiments on twelve representative methods confirm that having the right experts does not deliver the task, with a 48% drop in success once work turns interdependent. Trajectory analysis attributes the loss to three gaps: (1) **planning rigidity** in task orchestration, (2) **semantic compression** in information transfer, and (3) **adjudicative aggregation** in result integration. Even exhaustive collaboration or added memory cannot recover the loss. MAScope gives a common basis for measuring and **improving collaboration**, and is available at <https://anonymous.4open.science/r/MAScope-7B76>.

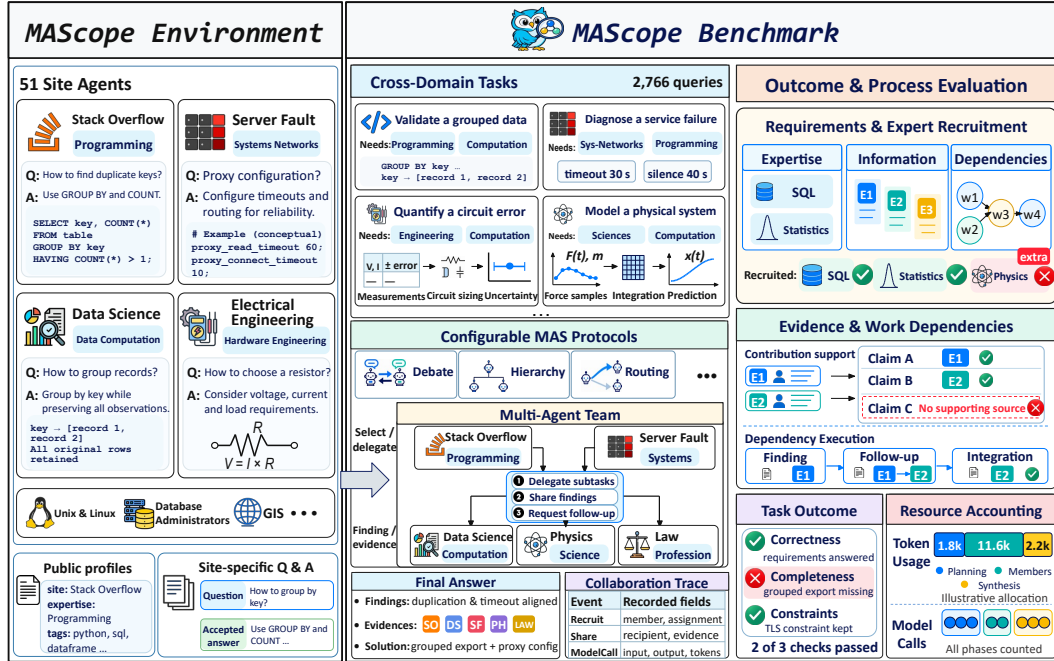


Figure 1: Overview of MAScope. The environment (left) provides 51 expert agents with public profiles and private Q&A corpora. The benchmark (right) lets MAS methods recruit agents and collaborate autonomously on 2,766 cross-domain tasks, and verifies expert recruitment, evidence support, work dependencies, and task outcome while accounting for resource use.

1 INTRODUCTION

Agent tasks are moving from closed-form question answering toward interactive work in real environments such as software development, data analysis, and tool use (Trivedi et al., 2024). These tasks require cross-domain knowledge, long-horizon state tracking, and adaptation to feedback (Yao et al., 2024; Ma et al., 2024). Multi-agent systems (MAS) meet these demands by distributing a task among agents with different expertise and solving it through communication and mutual verification, and are regarded as a promising organizational form for complex tasks (Li et al., 2023a).

However, gathering agents does not by itself create collaborative value. A larger team brings wider expertise, yet multiplies coordination overhead and opens more paths for errors to propagate (Kim et al., 2025). Working together, agents may misjudge their peers’ intentions (Li et al., 2023b), leave commitments unfulfilled and features conflicting in coding tasks (Khatua et al., 2026), and omit context, repeat reasoning, or aggregate prematurely in information-seeking tasks (Cemri et al., 2025). The same agents under different organizational structures yield markedly different results (Zhuge et al., 2024; Qian et al., 2025). A team’s value therefore lies in the collaboration itself. The central challenge is to *identify what kind of collaboration is valuable and where it breaks down*.

Existing domain benchmarks (Jimenez et al., 2024; Mialon et al., 2024; Center for AI Safety et al., 2026) evaluate the whole system as one method through its final outcome (Kapoor et al., 2025), leaving an internal failure difficult to interpret. Recent multi-agent benchmarks begin to measure the collaboration process through communication structure, coordination behavior, and framework-level capabilities (Zhu et al., 2025; Grötschla et al., 2025; Zhang et al., 2026; Orogat et al., 2026). Each measures collaboration from a single facet, and none follows a required contribution across the MAS workflow, from where it is demanded to where the outcome adopts it. Our preliminary study (§ 2) shows that this blind spot is harmful: (1) a MAS answers correctly without a required agent’s evidence, and (2) correct findings are discarded before reaching the answer. Attributing a MAS outcome to its collaboration therefore requires end-to-end evaluation across the whole workflow.

We introduce MAScope (§ 3) to make that evaluation possible. Our contributions are threefold:

- **An expert environment with disjoint private knowledge.** We build one expert agent per Stack Exchange community (Stack Exchange, Inc., 2014), 51 in all, each holding a private corpus no other agent can reach. Every passage returns with its source identifier, so a contribution stays traceable from the agent that retrieves it to the answer that carries it.
- **Collaborative tasks with certified work dependencies.** 2,766 queries from 499 task families span 11,018 required expertise units, crossing three dependency structures, *parallel*, *sequential*, and clue-based expert *discovery*, with three scales of required expertise. A solve-based test certifies the dependency graph of every dependent query, and every correct solution traverses it.
- **End-to-end evaluation of the collaboration workflow.** We decompose the workflow into four measurable capabilities, *expert selection*, *task orchestration*, *information transfer*, and *result integration*, and decide each one from the logged trajectory against a frozen reference. Two methods with the same pool and budget then separate at the stage where their organization differs.

With MAScope, we evaluate twelve influential MAS methods across the three coordination classes (§ 4). Having the right experts present does not deliver the task. They average only 41.3% success, a quarter of them fall short of a single agent holding the same corpora, and success drops a further 48% once work turns interdependent. Tracing every preregistered dependency shows the expertise is present throughout, since a recruited agent almost always solves the work it is given, and locates the loss in three gaps between agents, ① **planning rigidity** in task orchestration, ② **semantic compression** in information transfer, and ③ **adjudicative aggregation** in result integration. Collaborating to exhaustion and attaching a memory module both leave these gaps in place, each buying back a small fraction of what dependency costs. What limits a MAS is therefore the mechanism organizing the work, and MAScope makes each of these gaps measurable and comparable.

2 MOTIVATIONS AND OBSERVATIONS

We first examine whether the final outcome of a MAS reflects the collaboration behavior behind it. We run three representative methods, AutoGen (Wu et al., 2024), CrewAI (Moura, 2023), and

Anemol (Ren et al., 2025), on multi-hop question answering (MuSiQue, Trivedi et al., 2022) and general assistant tasks (GAIA, Mialon et al., 2024), with a shared backbone, four agents, and a budget of 150k tokens per task, averaged over five runs. For every run we freeze all agent records before aggregation and re-aggregate twice under the same rule, (I) from the complete record and (II) after masking the evidence contributed by one required agent (Appendix E). The same trajectories yield two observations about the MAS collaboration process:

Observation 1: A MAS answers correctly without a required agent’s evidence. Masking one required agent lowers accuracy by only 12.5% on MuSiQue but by 48.7% on GAIA (Figure 2, left). The gap arises because MuSiQue evidence is textual and the aggregating agent can reason it out from the remaining context and its own knowledge, whereas GAIA evidence comes from file processing and web retrieval and cannot be reproduced by reasoning alone. Evaluation on the outcome therefore credits the team for work the aggregator did alone, and the size of that credit shifts severalfold across environments while the outcome stays silent.

Observation 2: Correct findings are discarded before reaching the answer. Among failed runs, 29.8% on MuSiQue and 45.5% on GAIA already contain the decisive finding, produced by some agent but never carried into the answer (Figure 2, right). The failure therefore lies downstream of the finding, in the handoff and the aggregation, where a stronger agent has nothing to contribute, and evaluation on the outcome still records these runs as agent failures.

Together the two observations show that evaluation on the outcome cannot separate obtaining a contribution from using it. Evaluating collaboration therefore needs every required contribution to be indispensable, ruling out compensation, and traceable from production to adoption, exposing where it is lost.

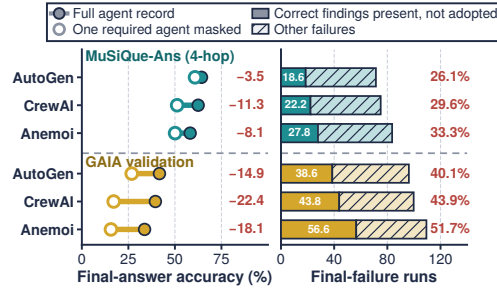


Figure 2: MAS collaboration behavior on MuSiQue and GAIA. We report final-answer accuracy with the full agent record and with one required agent masked (left), and failed runs split by whether a correct finding was already present (right).

3 MASCOPE: TRACING CONTRIBUTIONS TO THEIR ADOPTION

To evaluate multi-agent collaboration end to end, we develop MAScope, an environment where every required contribution is indispensable and can be followed from the agent that produces it to the answer that adopts it. First, we build an expert environment from 51 Stack Exchange communities and compose collaborative tasks on top of it (§ 3.1). Second, we organize these tasks along two independent dimensions, how their pieces of work depend on each other and how much expertise they require (§ 3.2). Third, we design an evaluation framework that follows each contribution through the collaboration workflow and measures four capabilities (§ 3.3).

3.1 BENCHMARK CONSTRUCTION

Expert environment. We instantiate one expert agent per Stack Exchange community, 51 in total. Each agent holds a public profile and a private corpus. MAS methods see only the profile, which lists the site name, expertise scope, and topic tags, when deciding whom to recruit. Execution draws on the corpus, which holds the community’s questions, accepted answers, and source identifiers for retrieval and citation. Disjoint corpora prevent one agent from supplying evidence that belongs to another, while identifiers minted inside the benchmark keep a memorized copy of Stack Exchange from passing as evidence. All agents share the same backbone and differ only in their corpora, so task completion depends entirely on how their information is organized.

Task definition. A task is a tuple $T = (q, U, G, V)$. The method sees only q , a user’s goal and observations stated without naming any site. Behind it, U enumerates the information units a solution requires, each bound to a source identifier, G records which units can be scoped only once a

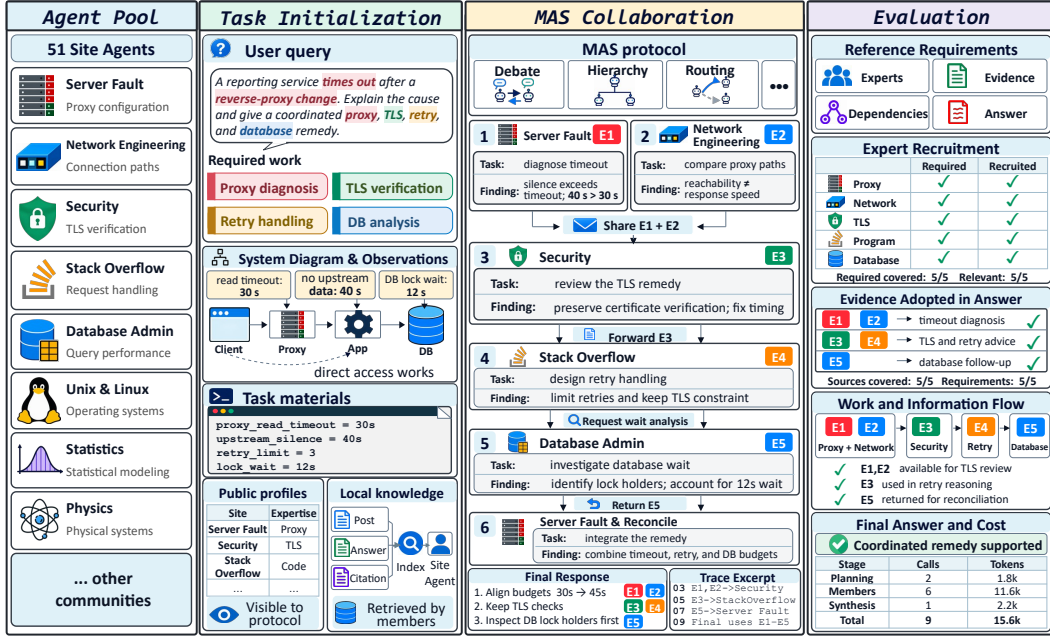


Figure 3: A MAScope run on a service-diagnosis task. The task needs five information units (E1–E5), the last three of which can be scoped only once the diagnosis in the first two is in hand. The evaluator checks recruited experts, whether each dependency was carried forward with its constraints, which evidence reached the answer, and the tokens each stage consumed.

predecessor finding is in hand, and V states the acceptance criteria. Tasks admit multiple valid paths, with equivalent evidence and alternative routes recorded in the reference, so evaluation presumes no fixed order of agent calls.

Construction pipeline. On top of this environment, each task is built in four steps.

- **Evidence alignment.** From real community questions and answers, we select units that belong to different sites yet serve one goal, verify each against its corpus, and bind it to a source identifier.
- **Task composition.** The demands of these units are rewritten into one user-facing question that gives the goal and the observed symptoms, leaving the required expertise to be inferred.
- **Dependency annotation.** A unit is dependent when its inputs or objectives can be fixed only from a predecessor finding. We confirm this by solving the unit with and without that finding, and record expert discovery when the finding also reveals the expertise needed next.
- **Validation and freezing.** To confirm that a task is well posed, we solve it end to end, checking that every corpus is reachable and the required expertise is inferable from q . Units whose evidence cannot be recovered from their own corpus are revised or dropped, tasks are frozen by family, and only q , the profiles, and the environment ship.

3.2 BENCHMARK OVERVIEW

Dataset statistics. MAScope contains 2,766 queries from 499 task families with 11,018 required expertise units. The 51 agents span six resource groups, from software engineering to professional practice, and a single task usually draws on more than one group, producing integrated goals in system diagnosis, data processing, engineering analysis, and professional judgment.

Task taxonomy. We characterize each task by its collaboration structure C and required expertise S . PARALLEL composition (C1) splits a task into independent pieces combined at the end, such as a transformation rule and a check for its result. SEQUENTIAL dependence (C2) chains them, so a later piece takes an earlier finding as its input, as when a configuration change follows from where a timeout originates. Clue-based expert DISCOVERY (C3) hides the next expert’s identity inside a predecessor finding, which the method must recognize while solving. S counts the professional

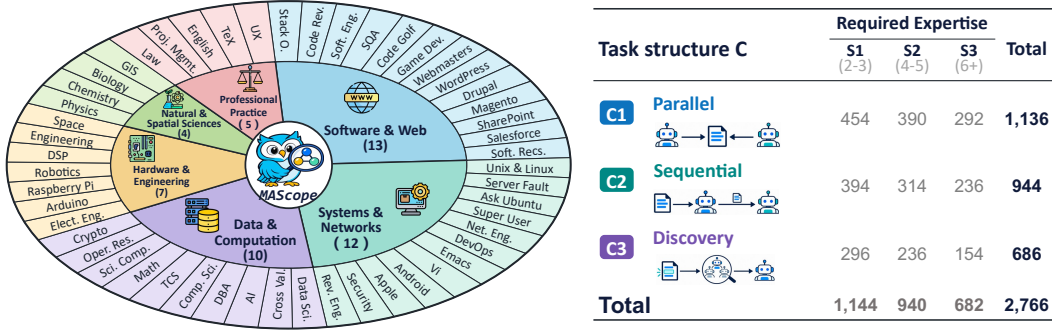


Figure 4: Expert pool and task taxonomy. MAScope holds 51 expert agents in six resource groups (left), and spreads its 2,766 queries over collaboration structure crossed with required expertise (right).

roles required, S1 (2–3 experts), S2 (4–5), and S3 (6+), without capping how many agents a method calls. As C and S vary independently, MAScope separates the effect of deeper dependency from that of broader expertise on the same method.

3.3 EVALUATION FRAMEWORK

Autonomous execution and outcome verification. A method receives q , the public profiles, a shared agent pool, tools, and a budget, and then recruits agents, assigns work, exchanges information, and submits a final answer entirely under its own mechanism, while the environment logs every agent call, message, local output, and token. The evaluator verifies the submitted answer against V to obtain task success, maps the evidence cited in the answer onto U to obtain evidence coverage, which records how much of the required content was completed even when the task fails, and sums tokens across planning, execution, aggregation, and memory operations to obtain execution cost.

Tracing contributions along the dependency chain. To locate where collaboration breaks down, we follow each required contribution along the chain that should carry it into the answer. For every C2 and C3 query we preregister its whole dependency graph G , whose edges every valid solution must traverse, and a method is ready to act on an edge once it holds the predecessor finding and has recruited the downstream expert ($s^{(1)}$). Along this chain, we measure four capabilities. (i) *Expert selection* covers the required expertise, measured over all queries by how completely and how precisely the participating agents A_q match the required experts R_q . (ii) *Task orchestration* turns the available finding into the next piece of work by assigning it to that expert ($s^{(2)}$), and (iii) *information transfer* delivers the assignment with the predecessor’s evidence and key constraints intact ($s^{(3)}$). The expert then produces a correct local result ($s^{(4)}$), which we record as local solve to separate an agent failing to solve from a team failing to adopt, and (iv) *result integration* carries that result into the final answer ($s^{(5)}$). Each capability is decided on every edge, and a query reaches a stage when all of its edges do. With $s_q^{(k)} \in \{0, 1\}$ for reaching stage k conditional on stage $k - 1$, the four capability rates are

$$r_k = \frac{\sum_q s_q^{(k)}}{\sum_q s_q^{(k-1)}}, \quad k = 2, \dots, 5.$$

Every stage is decided from the logged trajectory by matching source identifiers and preregistered terms, which keeps each capability comparable across coordination classes.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Compared methods. We evaluate twelve MAS methods grouped by how centralized their coordination is. (1) *Centralized* methods route every decision through one coordinator: AutoGen (Wu et al., 2024), LangGraph (Barda, 2025), CrewAI (Moura, 2023), Magentic-One (Fourney et al., 2024), and Flow (Niu et al., 2025). (2) *Semi-centralized* methods keep a planning, aggregation, or

Table 1: Main results on MAScope. We report task success (%) crossing **C1** parallel, **C2** sequential, and **C3** expert discovery with expertise S1 to S3. Background color marks the level of **degradation** from the C1 cell at the same scale, and Δ_{dep} and Δ_{scale} give the relative change from C1 to C2UC3 and from S1 to S3. $\pm$ is one standard deviation over five runs.

Method	C1 Parallel			C2 Sequential			C3 Discovery			Overall			Cost	
	S1	S2	S3	S1	S2	S3	S1	S2	S3	Success $\uparrow$	Δ_{dep}	Δ_{scale}	Tok (k) $\downarrow$	Calls $\downarrow$
Centralized														
AutoGen	74.3	65.7	53.9	57.6	44.0	28.1	49.6	34.4	20.6	52.1 ± 0.66	-35.9	-39.7	171	14.6
Magentic-One	72.5	64.9	52.7	53.4	41.0	24.7	48.4	33.5	21.0	50.2 ± 0.55	-38.3	-39.9	208	23.2
Flow	75.0	65.8	53.3	45.5	34.0	20.5	38.7	26.1	16.5	46.6 ± 0.39	-50.4	-39.4	126	14.4
CrewAI	77.2	68.9	55.3	43.2	27.0	15.8	33.6	21.1	12.1	44.9 ± 0.51	-58.9	-41.1	119	11.8
LangGraph	64.8	54.2	41.2	47.9	33.5	22.6	36.1	27.4	15.8	42.3 ± 0.30	-39.5	-43.6	127	16.7
Avg.	72.8	63.9	51.3	49.5	35.9	22.4	41.3	28.5	17.2	47.2 ± 0.21	-44.6	-40.8	150	16.1
Semi-centralized														
Anemol	64.3	57.4	54.2	42.6	33.4	25.3	39.3	31.3	17.8	44.3 ± 0.70	-43.1	-28.5	118	15.5
AgentVerse	67.6	60.2	48.8	39.8	27.3	16.4	37.7	25.3	15.1	41.9 ± 0.79	-51.5	-40.4	165	19.9
MoA	61.2	55.1	44.9	21.2	12.4	6.6	25.6	13.6	9.2	32.0 ± 0.19	-70.9	-38.2	1,044	94.6
MAD	52.2	42.6	30.9	25.9	18.7	11.2	26.9	17.7	8.2	29.4 ± 1.00	-54.6	-48.2	125	16.2
Avg.	61.3	53.8	44.7	32.4	23.0	14.9	32.4	22.0	12.6	36.9 ± 0.24	-55.0	-38.8	363	36.6
Decentralized														
GoA	65.6	62.9	60.5	43.3	36.3	29.4	36.4	27.5	19.7	46.2 ± 0.78	-46.1	-19.5	208	26.8
AgentNet	56.7	43.5	29.2	30.8	20.8	13.2	39.9	27.3	13.6	33.8 ± 0.51	-42.7	-53.5	88	7.9
SelfOrg	50.5	43.5	42.7	29.4	19.4	19.2	27.0	20.5	11.8	32.3 ± 0.49	-51.0	-25.8	324	104.3
Avg.	57.6	50.0	44.1	34.5	25.5	20.6	34.4	25.1	15.1	37.4 ± 0.34	-46.6	-33.0	206	46.3
Single Agent	54.7	43.2	32.3	32.9	23.6	14.8	28.9	19.6	13.8	32.6 ± 0.40	-46.5	-45.6	131	11.9

evaluation role while letting agents answer in parallel or talk directly: Anemol (Ren et al., 2025), AgentVerse (Chen et al., 2024), MoA (Wang et al., 2025), and MAD (Du et al., 2023). (3) *Decentralized* methods have no fixed coordinator and let agents propagate information or route work themselves: GoA (Yun et al., 2026), AgentNet (Yang et al., 2025), and SelfOrg (Tastan et al., 2026). Each runs under its native mechanism and decides for itself whom to recruit, how to organize the work, and when to stop. As a no-collaboration reference we add a single agent with access to all 51 corpora, which holds the whole task in one execution context.

Implementation details. Every method and agent uses `claude-sonnet-5` (Anthropic, 2026) as its backbone and shares the same expert agents, profiles, and retrieval interface. Each task is capped at 2M tokens and 256 model calls, and we report the consumption actually incurred. All experiments run five times over the 2,766 queries. Details are in Appendix D.

4.2 MAIN RESULTS

① **A MAS does not reliably outperform a single agent.** Nine methods clear the single agent’s 32.6%, led by AutoGen at 52.1% and trailed by AgentNet at 33.8%, while the other three fall below the single agent outright, as low as 29.4%, even though MoA and SelfOrg put all 51 experts to work. With corpora, budget, and backbone held fixed, a MAS is separated from the single agent only by how it organizes the work after dividing it.

② **Complex work dependencies systematically degrade system performance.** When a MAS takes on tasks whose later work builds on earlier findings, average success falls from 57.8% to 29.8%. Every coordination class loses at least 44% of its success, so no organizational style is immune. Scaling the required expertise from 2–3 to 6+ experts cuts success by a further 38%. MoA and SelfOrg activate the entire pool and never miss a required expert, yet both trail the single agent on sequential-dependence tasks. The loss thus falls on the step that turns one agent’s finding into the next agent’s work, not on the availability of expertise.

Table 2: Collaboration capabilities along the dependency chain (%). We report expert selection over all queries and the remaining capabilities as conditional rates on the 1,630 dependent queries, each scored on every edge of its preregistered graph. Background color marks how far a method **trails** its column’s leader.

Method	Expert Selection		Collaboration Pipeline				Overall	
	Recall	Precision	Task Orchestration	Information Transfer	Local Solve	Result Integration	Dep. Survival	Evidence Coverage
Centralized								
AutoGen	81.8	74.0	85.5	89.8	91.6	92.3	64.9 $\pm$ 1.06	67.4
Magentic-One	80.4	71.6	86.5	83.5	91.3	92.2	60.8 $\pm$ 0.66	65.7
Flow	83.1	71.9	83.2	85.4	90.4	90.1	57.9 $\pm$ 0.72	63.6
CrewAI	81.9	73.7	70.9	81.8	89.4	91.9	47.6 $\pm$ 0.88	61.8
LangGraph	77.7	68.6	77.3	85.3	89.7	94.4	55.8 $\pm$ 0.79	58.1
Avg.	81.0	72.0	80.7	85.2	90.5	92.2	57.4 $\pm$ 0.53	63.3
Semi-centralized								
Anemioi	78.9	69.4	92.0	80.5	93.1	76.4	52.6 $\pm$ 1.15	58.5
AgentVerse	86.6	66.7	76.0	77.0	90.6	79.8	42.3 $\pm$ 0.60	58.7
MoA	100.0	7.8	34.8	77.7	90.4	80.0	19.6 $\pm$ 0.25	72.1
MAD	65.1	70.0	87.8	83.3	87.6	73.9	47.3 $\pm$ 1.18	44.0
Avg.	82.6	53.5	72.6	79.6	90.4	77.5	40.5 $\pm$ 0.34	58.3
Decentralized								
GoA	84.1	69.6	91.7	71.7	88.4	87.3	50.8 $\pm$ 0.49	62.6
AgentNet	68.1	87.0	87.6	79.3	90.5	85.9	54.0 $\pm$ 0.57	49.6
SelfOrg	100.0	7.8	85.5	65.6	86.9	81.2	39.6 $\pm$ 0.95	49.2
Avg.	84.1	54.8	88.3	72.2	88.6	84.8	48.1 $\pm$ 0.28	53.8

③ **Higher cost does not translate into better performance.** Per-task cost spans an order of magnitude, from 88k tokens for AgentNet to 1,044k for MoA, tracking how many agents answer, not how well. Success does not track cost. Flow reaches 46.6% at 126k tokens while MoA reaches 32.0% at 8 $\times$ the cost, and AgentNet, the cheapest, still edges past the single agent. The performance–cost frontier belongs to the centralized class, where AutoGen leads at 1.3 $\times$ the single agent’s cost. Extra computation buys participation and communication, not the missing piece of work.

4.3 WHY DOES COLLABORATION FAIL?

To locate where collaboration breaks down, we trace every preregistered dependency graph through the four capabilities of § 3.3. The results indicate that agents are reliable at completing work on their own, and hence reveal three capability gaps in the MAS collaboration process.

① **Agent findings struggle to revise an established plan.** In 20% of dependent queries a finding arrives but the work it implies is never assigned, and we call this **planning rigidity** in task orchestration. Centralized methods follow a decomposition-before-execution pipeline, so a finding produced at run time has no route back into the plan. CrewAI executes only predefined tasks and assigns 70.9% of the work its findings imply, and LangGraph 77.3%. Methods with a replanning trigger fare no better, as Magentic-One and Flow revise only when progress stalls or a step fails, reaching 86.5% and 83.2%. Orchestration holds up where an agent can claim work for itself, with Anemioi, GoA, and AgentNet between 87.6% and 92.0%. MoA is an extreme case at 34.8%, as its layers pass merged answers upward and assign follow-up work only where a merge happens to state it.

② **Agents tend to transfer conclusions without supporting evidence.** Of the work that is assigned, another 20% arrives as a bare conclusion stripped of its evidence and constraints, and we call this **semantic compression** in information transfer. Decentralized methods relay each finding hop by hop, and a restatement cannot carry what the next step has not asked for yet. SelfOrg and GoA broadcast round summaries and deliver usable inputs in 65.6% and 71.7% of cases, and AgentVerse, which compresses an assignment into one subtask sentence, reaches 77.0%. Transfer holds up wherever the predecessor’s output passes on intact, whether a coordinator forwards the full conversation, as in AutoGen at 89.8%, or a graph edge carries the output, as in Flow at 85.4%.

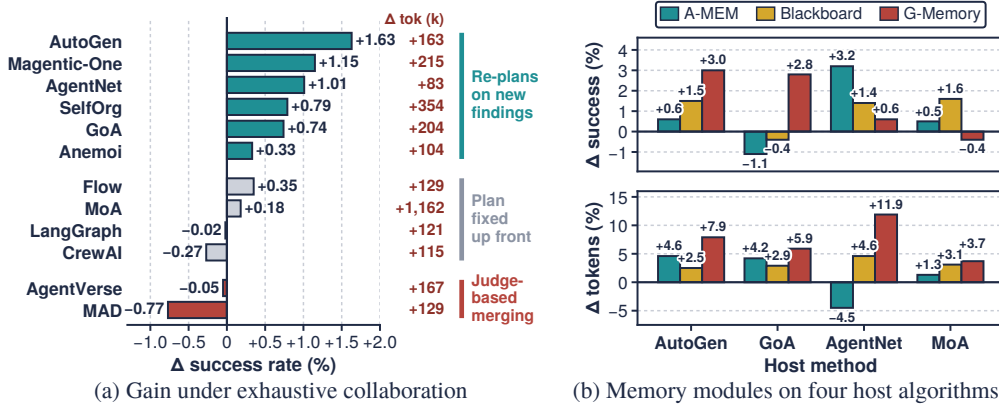


Figure 5: Exhaustive collaboration and memory modules. We report the change in success (a) when a method must collaborate until it confirms completion, grouped by mechanism and annotated with the extra tokens spent in red, and (b) when three memory modules are added to four host methods. Error bars are one standard deviation of the paired difference over five runs.

③ **Correct local results are overridden during aggregation.** Once a correct local result exists, 14.5% of the time it never reaches the answer, and we call this **adjudicative aggregation** in result integration. Semi-centralized methods treat agent outputs as competing answers and select among them, a rule that rewards agreement over correctness and drops a correct result held by one agent. MAD retains 73.9% and Anemoi 76.4%, and moving the decision to another role does not change it. Centralized methods assemble outputs into one answer instead, and all retain above 90%.

Overall, each MAS class carries its own strength and its own blind spot. In centralized methods a coordinator holds the full context and assembles the answer, which secures transfer and integration but fixes the plan before any finding exists. Decentralized methods remove that coordinator so any agent can pick up new work, yet the hop-by-hop relay that makes this possible thins the evidence along the way. Between the two, semi-centralized methods buy parallel progress with an arbitration step that can overwrite a result already correct.

4.4 CAN EXHAUSTIVE COLLABORATION RECOVER PERFORMANCE?

To test whether more collaboration recovers these gaps, we force each method to keep going under its own mechanism until it confirms that every requirement is met and no agent can contribute further.

① **Exhaustive collaboration does not recover MAS performance.** Collaborating to exhaustion improves success by at most 1.6%, lowers it for three methods, and leaves three inside their run-to-run spread, while continuing costs between $1.3\times$ and $3.3\times$ the native run. Set against the 28-point gap between parallel and dependency tasks, even the best of these gains recovers under 6% of it. The extra rounds go into re-running the same plan, regenerating the same answer, or adjudicating again, so work a method never arranged before remains unarranged afterwards.

② **Only methods that turn new findings into new work benefit.** Every method that lets findings re-enter planning gains, from 0.33% for Anemoi, the one inside its spread, to 1.63% for AutoGen, because continuing means revisiting what has been found and dispatching what follows from it. Methods whose plan is fixed up front stay between -0.27% and $+0.18\%$, since another round executes the same plan and any gain is offset by fresh errors. Both methods that merge by adjudication decline, AgentVerse by 0.59% and MAD by 0.77%, as every additional round is another chance to overwrite a result that was already correct. The methods carrying the largest orchestration gap are therefore the ones least able to use the extra collaboration.

4.5 COMBINING MEMORY MODULES WITH MAS METHODS

Memory supplies the experience a MAS can reuse when selecting experts, making later decisions, and reusing information, which makes it another common enhancement. We adopt three influential modules into MAScope to quantify the value of combining a MAS with memory. A-MEM (Xu et al.,

2025) keeps cross-task experience as structured notes on each agent, Blackboard (Han & Zhang, 2025) holds a shared working state that the team reads and writes within a task, and G-Memory (Zhang et al., 2025a) retrieves across tasks and steps through a three-level graph of experience, tasks, and interactions. We pair each module with AutoGen, GoA, AgentNet, and MoA, four representative methods drawn from the three coordination classes.

① **A memory module helps only where it supplies what the host lacks.** No module helps every host, and the best module differs across hosts. AgentNet routes work through local decisions taken by each agent, the level at which A-MEM keeps its notes, and the match yields the largest gain of 3.2%. G-Memory’s cross-step retrieval addresses the dependent work that AutoGen and GoA fail to connect, giving 3.0% and 2.8%, and Blackboard’s shared state lets MoA agents see one another’s work, worth 1.7%. The reverse pairings fail. Cross-step retrieval leaves MoA inside its own spread because MoA does no cross-step work, while agent notes and shared state cost GoA 1.1% and 0.4% because its losses come from relaying summaries.

② **Memory gains are usually paid for in tokens.** Retrieval and writing add 1% to 12% across most pairings, and the additions are largest where gains are smallest, as with G-Memory on AgentNet at 11.9% more tokens for 0.6% more success. AgentNet with A-MEM is the one pairing that pays for itself, cutting tokens by 4.5% while raising success by 3.2%, since knowing what each agent does well removes routing attempts that lead nowhere. Experience that enters a decision saves the trial and error it replaces, and experience that does not leaves only the cost of retrieval.

5 RELATED WORK

MAS methods. MAS methods differ mainly in who decides how work is organized. Centralized methods route that decision through a coordinator, evolving from fixed roles and standard procedures to speaker selection, shared-state routing, and dispatch over subtask graphs (Hong et al., 2024; Wu et al., 2024; Fournay et al., 2024; Niu et al., 2025; Barda, 2025). Semi-centralized methods keep a planning, aggregation, or evaluation role while agents answer in parallel or talk to each other directly (Du et al., 2023; Wang et al., 2025; Ren et al., 2025). Decentralized methods drop the coordinator and let agents propagate information or route work themselves (Zhuge et al., 2024; Qian et al., 2025; Yun et al., 2026; Yang et al., 2025), and a further line searches for the organization automatically (Zhang et al., 2025c;b). Such diversity leaves each mechanism buried in a single final outcome, so MAScope evaluates all of them in one environment and attributes their differences stage by stage.

Evaluation of multi-agent collaboration. General agent benchmarks evaluate a system by what it finally completes (Mialon et al., 2024; Jimenez et al., 2024; Wu et al., 2026). Multi-agent benchmarks open the process up, measuring milestones and communication structure (Zhu et al., 2025), distributed reasoning on prescribed topologies (Grötschla et al., 2025), coordination under information silos (Zhang et al., 2026), framework-level capabilities (Orogat et al., 2026), and teamwork under enforced role separation (Kim et al., 2026), while a parallel line reads trajectories for failure taxonomies (Cemri et al., 2025), scaling conditions (Kim et al., 2025), and agent credit (Jiang et al., 2026). These lines each open a different facet of the process, and none follows a required contribution across the MAS workflow, from the demand that calls for it to the answer that adopts it. MAScope makes every contribution indispensable through private corpora and verifiable through hidden references, so a difference in outcome can be traced back and compared quantitatively.

6 CONCLUSION

We introduce MAScope, an environment of 51 expert agents with disjoint private knowledge that traces every required contribution from production to adoption. Experiments on twelve influential MAS methods show that having the required experts does not deliver the task. Success halves once work becomes interdependent, and three gaps account for it: planning rigidity in task orchestration, semantic compression in information transfer, and adjudicative aggregation in result integration. Even exhaustive collaboration or added memory cannot recover the loss. MAScope turns these gaps into verifiable steps, so that a proposed improvement can be located and tested stage by stage.

REPRODUCIBILITY STATEMENT

Acceptance in MAScope is settled by string against a frozen reference, so re-running the evaluator over a stored trajectory returns the verdicts we report, and the rates follow from them. Every step that produces those trajectories is specified in the appendix. Appendix A gives the 51 communities, the filtering that yields each corpus, and the two surfaces an agent works through. Appendix B gives the construction pipeline, the solve-based protocol that certifies a dependency, and the checks a task clears before it is frozen. Appendix C gives the evaluation protocol, from the event stream an adapter produces to the five predicates that decide a preregistered dependency. Appendix D gives the twelve methods with their configurations, the single-agent reference and the two enhancement settings, and Appendix H prints the three prompts MAScope adds. Appendix F reports the counts behind every rate together with its run-to-run spread, and Appendix I states what the release ships and under which licenses.

AI USE STATEMENT

We used generative AI tools as a writing and presentation aid, to edit and polish prose written by the authors and to prepare the scripts that render our measured results into figures and tables. The authors reviewed all AI-assisted output and take responsibility for this paper.

REFERENCES

- Anthropic. Introducing Claude Sonnet 5. Anthropic, June 2026. URL <https://www.anthropic.com/news/claude-sonnet-5>. Accessed: 2026-09-16.
- Vadym Barda. LangGraph Multi-Agent Supervisor. Software, 2025. URL <https://github.com/langchain-ai/langgraph-supervisor-py>. Accessed: 2026-09-16.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems*, volume 38, Main Conference. Curran Associates, Inc., 2025. doi: 10.52202/085713-4082. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/b1041e52d3be19f0a9bc491657488e4a-Abstract-Datasets_and_Benchmarks_Track.html.
- Center for AI Safety, Scale AI, and HLE Contributors Consortium. A benchmark of expert-level academic questions to assess AI capabilities. *Nature*, 649(8099):1139–1146, 2026. doi: 10.1038/s41586-025-09962-4. URL <https://www.nature.com/articles/s41586-025-09962-4>.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (eds.), *International Conference on Learning Representations*, volume 2024, pp. 20094–20136, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/578e65cdee35d00c708d4c64bce32971-Paper-Conference.pdf.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL <https://arxiv.org/abs/2305.14325>.
- Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang (Eric) Zhu, Friederike Niedtner, Grace Proebsting, Griffin Gorman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-One: A generalist multi-agent system for solving complex tasks, 2024. URL <https://arxiv.org/abs/2411.04468>.

- Florian Grötschla, Luis Müller, Jan Tönshoff, Mikhail Galkin, and Bryan Perozzi. AgentsNet: Coordination and collaborative reasoning in multi-agent LLMs, 2025. URL <https://arxiv.org/abs/2507.08616>.
- Bochen Han and Songmao Zhang. Exploring advanced LLM multi-agent systems based on black-board architecture, 2025. URL <https://arxiv.org/abs/2507.01701>.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (eds.), *International Conference on Learning Representations*, volume 2024, pp. 23247–23275, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/6507b115562bb0a305f1958ccc87355a-Paper-Conference.pdf.
- Pengyi Jiang, Xiaoguang Zhu, and Quanyan Zhu. Semantic cooperative games for contribution attribution in LLM-based multi-agent systems, 2026. URL <https://arxiv.org/abs/2607.18255>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, pp. 54107–54157, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html.
- Sayash Kapoor, Benedikt Stroebel, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=Zy4uFzMviZ>.
- Arpandeeep Khatua, Hao Zhu, Peter Tran, Arya Prabhudesai, Frederic Sadrieh, Johann K. Lieberwirth, Xinkai Yu, Yicheng Fu, Michael J. Ryan, Jiaxin Pei, and Diyi Yang. CooperBench: Why coding agents cannot be your teammates yet, 2026. URL <https://arxiv.org/abs/2601.13295>.
- Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Yun Liu, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. Towards a science of scaling agent systems, 2025. URL <https://arxiv.org/abs/2512.08296>.
- Yubin Kim, Chanwoo Park, Taehan Kim, Eugene Park, Samuel Schmidgall, Salman Rahman, Chunjong Park, Cynthia Breazeal, Xin Liu, Hamid Palangi, Hae Won Park, and Daniel McDuff. TeamBench: Evaluating agent coordination under enforced role separation, 2026. URL <https://arxiv.org/abs/2605.07073>.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "Mind" exploration of large language model society. In *Advances in Neural Information Processing Systems 36*, 2023a. doi: 10.52202/075280-2264. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/a3621ee907def47c1b952ade25c67698-Abstract-Conference.html.
- Huaoli, Yu Quan Chong, Simon Stepputtis, Joseph Campbell, Dana Hughes, Michael Lewis, and Katia Sycara. Theory of mind for multi-agent collaboration via large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 180–192, Singapore, December 2023b. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.13. URL <https://aclanthology.org/2023.emnlp-main.13/>.
- Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37, pp. 74325–74362. Curran Associates, Inc., 2024. doi: 10.52202/079017-2365. URL https://proceedings.neurips.cc/paper_files/

- paper / 2024 / file / 877b40688e330a0e2a3fc24084208dfa - Paper - Datasets_and_Benchmarks_Track.pdf.
- Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. In *International Conference on Learning Representations*, pp. 9025–9049, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/25ae35b5b1738d80f1f03a8713e405ec-Abstract-Conference.html.
- João Moura. CrewAI. Software, 2023. URL <https://github.com/crewAIInc/crewAI>. Accessed: 2026-09-16.
- Boye Niu, Yiliao Song, Kai Lian, Yifan Shen, Yu Yao, Kun Zhang, and Tongliang Liu. Flow: Modularized agentic workflow automation. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 74949–74977, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/ba84da6921f3040b74ee163aa7451f53-Paper-Conference.pdf.
- Abdelghny Orogat, Ana Rostam, and Essam Mansour. Understanding multi-agent LLM frameworks: A unified benchmark and experimental analysis, 2026. URL <https://arxiv.org/abs/2602.03128>.
- Chen Qian, Zihao Xie, YiFei Wang, Wei Liu, Kunlun Zhu, Hanchen Xia, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Scaling large language model-based multi-agent collaboration. In *International Conference on Learning Representations*, pp. 41488–41505, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/66a026c0d17040889b50f0dfa650e5e0-Abstract-Conference.html.
- Xinxing Ren, Caelum Forder, Qianbo Zang, Ahsen Tahir, Roman J. Georgio, Suman Deb, Peter Carroll, Önder Gürçan, and Zekun Guo. Anemol: A semi-centralized multi-agent system based on agent-to-agent communication MCP server from coral protocol, 2025. URL <https://arxiv.org/abs/2508.17068>.
- Stack Exchange, Inc. Stack Exchange Data Dump. Internet Archive, 2014. URL <https://archive.org/details/stackexchange>. Dataset; accessed September 16, 2026.
- Nurbek Tastan, Samuel Horváth, and Karthik Nandakumar. Stochastic self-organization in multi-agent systems. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 40242–40271, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/43833a8a514392cb19ca09b52cf4b5b7-Paper-Conference.pdf.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022. doi: 10.1162/tacl_a_00475. URL <https://aclanthology.org/2022.tacl-1.31/>.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.850. URL <https://aclanthology.org/2024.acl-long.850/>.
- Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Y Zou. Mixture-of-Agents enhances large language model capabilities. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 33944–33963, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/5434be94e82c54327bb9dcdf7fca52b6-Paper-Conference.pdf.

- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *COLM 2024*, 2024. URL <https://arxiv.org/abs/2308.08155>.
- Zijian Wu, Xiangyan Liu, xinyuan zhang, Lingjun Chen, Fanqing Meng, Lingxiao Du, Yiran Zhao, Fanshi Zhang, Yaoqi Ye, Jiawei Wang, Zirui Wang, Jinjie Ni, Yufan Yang, Arvin Xu, and Michael Qizhe Shieh. MCPMark: A benchmark for stress-testing realistic and comprehensive MCP use. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 79852–79895, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/8138d211ce8790fdfbbeeb9781838a37-Paper-Conference.pdf.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for LLM agents. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (eds.), *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pp. 17577–17604. Curran Associates, Inc., 2025. doi: 10.52202/085713-0593. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/19909c36f51abc4856b4560aff3d36d6-Paper-Conference.pdf.
- Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. AgentNet: Decentralized evolutionary coordination for LLM-based multi-agent systems. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (eds.), *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pp. 107309–107336. Curran Associates, Inc., 2025. doi: 10.52202/085713-3578. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/9a379c1b05793d1c42dc832269834515-Paper-Conference.pdf.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL <https://arxiv.org/abs/2406.12045>.
- Sukwon Yun, Jie Peng, Pingzhi Li, Wendong Fan, Jie Chen, James Y Zou, Guohao Li, and Tianlong Chen. Graph-of-Agents: A graph-based framework for multi-agent LLM collaboration. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 19745–19760, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/21a87fb07e9ab0ecc4d9d1b940676229-Paper-Conference.pdf.
- Guibin Zhang, Muxin Fu, Kun Wang, Guancheng Wan, Miao Yu, and Shuicheng Yan. G-Memory: Tracing hierarchical memory for multi-agent systems. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (eds.), *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pp. 12988–13018. Curran Associates, Inc., 2025a. doi: 10.52202/085713-0439. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/136a45cd9b841bf785625709a19c6508-Paper-Conference.pdf.
- Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent Architecture Search via Agentic Supernet. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 75834–75852. PMLR, 2025b. URL <https://proceedings.mlr.press/v267/zhang25bi.html>.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, XiongHui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating Agentic Workflow Generation. In *International Conference on Learning Representations*, volume 2025, pp. 34040–34077, 2025c. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/5492ecbce4439401798dcd2c90be94cd-Paper-Conference.pdf.
- Yuzhe Zhang, Feiran Liu, Yi Shan, Xinyi Huang, Xin Yang, Yueqi Zhu, Xuxin Cheng, Cao Liu, Ke Zeng, Terry Jingchen Zhang, and Wenyan Jiang. SILO-BENCH: A scalable environment

for evaluating distributed coordination in multi-agent LLM systems. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 29379–29398, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.1354. URL <https://aclanthology.org/2026.acl-long.1354/>.

Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. MultiAgentBench: Evaluating the collaboration and competition of LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8580–8622, Vienna, Austria, July 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.421. URL <https://aclanthology.org/2025.acl-long.421/>.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. GPTSwarm: Language agents as optimizable graphs. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 62743–62767. PMLR, 2024. URL <https://proceedings.mlr.press/v235/zhuge24a.html>.

Appendix Contents

A Expert Environment	16
A.1 Communities and Corpora	16
A.2 Public Profile and Private Retrieval	16
A.3 Source Identifiers and Memorization	17
B Task Construction	18
B.1 From Community Posts to a Composed Task	18
B.2 Certifying a Task’s Dependency Graph	19
B.3 Quality Control and Family Structure	20
B.4 Dataset Composition	20
C Evaluation Protocol	21
C.1 Execution and Logging	21
C.2 Outcome Verification	22
C.3 Expert Selection	23
C.4 Stage Decisions along the Dependency Chain	23
D Compared Methods	24
D.1 Integration	24
D.2 The Single Agent Reference	24
D.3 Exhaustive Collaboration	25
D.4 Memory Modules	26
E Preliminary Study	26
E.1 Tasks and Settings	26
E.2 Masking a Required Contribution	26
E.3 Locating a Discarded Finding	27
F Full Results	28
F.1 Where Dependent Queries Go	28
F.2 Exhaustive Collaboration	28
F.3 Memory Modules	28
F.4 The No-Collaboration Reference	30
G Qualitative Examples	31
G.1 One Task in Full	31
G.2 Planning Rigidity	31
G.3 Semantic Compression	31
G.4 Adjudicative Aggregation	33
H Prompts	34
I Limitations and Data	35
I.1 What the Findings Cover	35
I.2 What the Environment Leaves Out	36
I.3 Data and Release	36

Table 3: The 51 expert communities. Each agent is built from one Stack Exchange site, and the resource groups organize the pool for reporting. Tasks cross the grouping freely, so a single query may draw on one group or on five.

Resource group	Sites	Communities
■ Software & Web	13	Stack Overflow, Code Review, Software Engineering, SQA, Code Golf, Game Development, Webmasters, WordPress, Drupal, Magento, SharePoint, Salesforce, Software Recommendations
■ Systems & Networks	12	Unix & Linux, Server Fault, Ask Ubuntu, Super User, Network Engineering, DevOps, Emacs, Vi, Android, Apple, Security, Reverse Engineering
■ Data & Computation	10	Data Science, Cross Validated, Artificial Intelligence, Database Administrators, Computer Science, Theoretical CS, Mathematics, Scientific Computing, Operations Research, Cryptography
■ Hardware & Eng.	7	Electrical Engineering, Arduino, Raspberry Pi, Robotics, Signal Processing, Engineering, Space Exploration
■ Professional Practice	5	Law, Project Management, English Language, T _E X, User Experience
■ Natural & Spatial Sci.	4	Physics, Chemistry, Biology, Geographic Information Systems

A EXPERT ENVIRONMENT

MAScope replaces the homogeneous agent pool of earlier benchmarks with 51 experts whose knowledge does not overlap. This section gives the communities behind them (§ A.1), the two surfaces a method and an agent work through (§ A.2), and the mechanism that keeps a memorized copy of Stack Exchange from passing as evidence (§ A.3).

A.1 COMMUNITIES AND CORPORA

Each expert agent is built from one Stack Exchange community (Stack Exchange, Inc., 2014), taken from the data dump of 1 March 2026. We keep a question–answer pair when the question has an accepted answer, a body of at least 80 tokens, and at least one topic tag. The first condition gives every retained pair a community-endorsed resolution, and the other two remove stubs that carry no retrievable content. We then remove near-duplicates within a community, treating two pairs as duplicates when the MinHash similarity of their question bodies is at least 0.90 and keeping the one whose accepted answer has the higher score, so that a single widely reposted issue cannot be retrieved as several independent findings.

Retained pairs are capped at 20,000 per community, sampled uniformly at random. Without the cap Stack Overflow alone would hold two orders of magnitude more content than the smallest community, making one agent a near-universal oracle and turning recruitment into a question about corpus size. After capping, the pool holds 766,930 pairs, with a median of 18,271 per community and a minimum of 3,247. Twenty-three of the 51 corpora sit at the cap. Every retained pair is stored with the identifier introduced in § A.3. Table 3 lists the 51 communities in the six resource groups used throughout the paper, and Table 4 reports the size and the pair length of the resulting corpora.

The six groups describe the pool. A task may draw on one group or on five, and Figure 4 shows how the 2,766 queries spread over structure and scale once composed.

A.2 PUBLIC PROFILE AND PRIVATE RETRIEVAL

An agent exposes two surfaces, and the boundary between them is what turns recruitment into a measurable decision. Figure 6 shows both on one agent.

Public profile. The profile is the only thing a MAS method reads before recruiting. It carries the fields in Table 5 and never contains an answer, so recruiting well requires inferring the needed expertise from the question itself. The expertise scope is one sentence that states both what the agent

Table 4: Corpus statistics by resource group. We report the question–answer pairs each group retains after filtering and near-duplicate removal, under a cap of 20,000 pairs per community that 23 of the 51 corpora reach. The bar carries the group total against the largest group. Lengths are question plus accepted answer in backbone tokens, and the per-site columns are medians and ranges over the communities of a group.

Resource group	Sites	Q–A pairs	Median / site	Min–max / site	Median length
■ Software & Web	13	214,076	20,000	4,412–20,000	731
■ Systems & Networks	12	181,345	18,421	3,914–20,000	629
■ Data & Computation	10	134,709	13,480	3,247–20,000	669
■ Hardware & Eng.	7	86,440	9,832	5,274–20,000	671
■ Professional Practice	5	79,602	18,271	6,428–20,000	636
■ Natural & Spatial Sci.	4	70,758	18,921	12,916–20,000	673
Total	51	766,930	18,271	3,247–20,000	671

Table 5: Fields of a public profile. These four fields are everything a method knows about an agent when it decides whom to recruit, and none of them carries an answer.

Field	Content
Site name	The community the agent is built from, for example <i>Server Fault</i> .
Expertise scope	One sentence stating what the agent covers and what falls outside it.
Topic tags	The twelve most frequent tags of the retained corpus, sketching its sub-areas.
Corpus size	The number of question–answer pairs the agent can retrieve from.

covers and what falls outside it, and the topic tags are the twelve most frequent tags of the retained corpus, which sketch its sub-areas without naming any individual post.

Retrieval interface. During execution an agent queries its own corpus in natural language and receives at most eight passages, each a retained question–answer pair returned with its source identifier. An agent cannot query another agent’s corpus, and nothing in the interface reveals which other experts have been recruited or how many agents the team holds. Retrieval is the only way evidence enters a trajectory, and every call is charged to the task budget, so retrieval competes with the rest of the run for the same allowance. Whatever a method learns about its teammates it learns from their messages, which is what § 3.3 measures as information transfer.

A.3 SOURCE IDENTIFIERS AND MEMORIZATION

Stack Exchange is almost certainly in the pretraining data of any current backbone. Our design makes memorization insufficient by construction.

Every information unit in U is bound to an identifier that we mint inside the benchmark. An identifier is the prefix `MS-` followed by twelve characters of Crockford’s base-32 alphabet, derived from a keyed hash of the community, the post, and the unit index, under a key generated once at construction time and never released. Two properties follow. The identifier postdates any pretraining corpus, so no backbone holds it in its weights, and it depends on a key that never leaves construction time, so a memorized copy of the post gives no way to reconstruct it. Acceptance under V requires the final answer to carry the identifier of each required unit, which means a backbone that has memorized the underlying content still cannot produce an accepted answer without an agent retrieving from the corpus first. Memorization may help an agent phrase a finding. It cannot manufacture the token that proves the finding was obtained.

Empirical check. We verify this directly. From the 2,766 queries we draw 270, stratified over the nine $C \times S$ cells in proportion to cell size so that the subsample carries the same difficulty mix as the benchmark, and run AutoGen on them twice, once as in the main experiments and once with the corpus of one required agent made unreachable, chosen uniformly among that query’s required experts, with every other condition unchanged. The required expertise is still present in the pool and

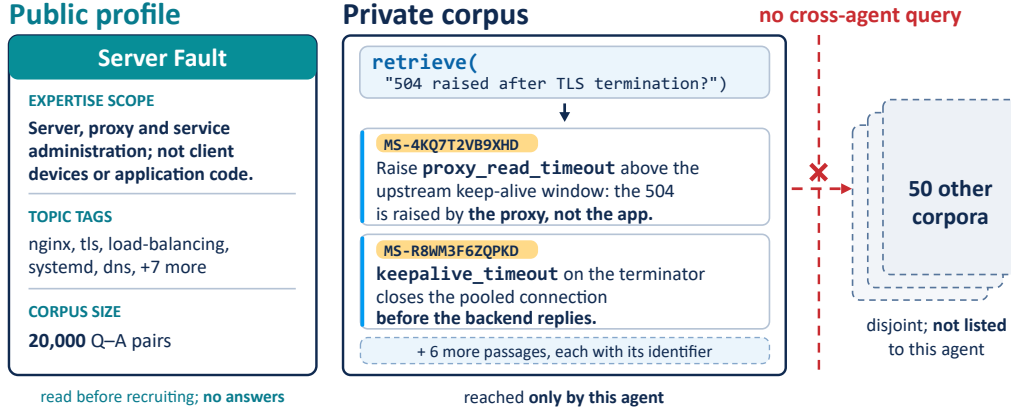


Figure 6: The two surfaces of one expert agent. A method reads the public profile (**left**) before recruiting, and the profile contains no answer. The recruited agent reaches its own corpus (**middle**) through a natural-language call that returns passages carrying benchmark-internal identifiers. No retrieval call reaches another agent’s corpus (**right**), so evidence held by one expert can enter the answer only if the team carries it there.

the agent still joins the team, and only its evidence is gone. Averaged over five runs, task success falls from 52.9% to 4.7%, or 142.8 to 12.6 of 270 queries, and the unmodified condition reproduces the 52.1% of Table 1 to within the noise of a 270-query subsample. Every success under removal comes from a query whose reference records an equivalent unit in another recruited corpus, so an alternative route covers the missing evidence, and no accepted answer in this condition cites an identifier from the removed corpus. Evidence therefore enters a trajectory through the corpus alone.

Licensing and anonymity. Stack Exchange content is released under CC BY-SA (Stack Exchange, Inc., 2014). We redistribute derived corpora under the same terms and keep the attribution the license requires, shipping the community, post identifier and revision date of every retained pair so each can be traced to its source. User names, profile links and other author-identifying metadata are stripped before a corpus enters the environment, so an agent never sees who wrote it.

B TASK CONSTRUCTION

Two properties define a MAScope task. Every required unit is reachable from exactly one corpus, and every dependency among units holds for any correct solution, whatever wording the question uses. Construction enforces both. This section gives the pipeline that produces a task (§ B.1), the protocol that certifies a task’s whole dependency graph (§ B.2), the checks a task clears before it is frozen (§ B.3), and the composition of the released set (§ B.4).

B.1 FROM COMMUNITY POSTS TO A COMPOSED TASK

Construction runs as a pipeline of four stages, each closing on a gate that discards the candidates failing it. Figure 7 traces one task through all four and records how many candidate groups each gate lets through.

Selecting candidate units. A candidate group starts from one accepted answer and grows by retrieving, from the other 50 corpora, answers whose content a single user goal would also require. A group is kept when its members come from at least two communities and none of them subsumes another. Subsumption is decided by solving. A unit u_j is subsumed by u_i when u_j is accepted with only the passages supporting u_i in context, so u_j adds no expertise to the group.

Composing the question. The units of a group are rewritten into one question that states a goal and the symptoms a user would observe, names no community, and uses no vocabulary confined to a single corpus. Composition is model-assisted, and two of the authors review every composed question independently against a four-item checklist, with a third resolving disagreements. The

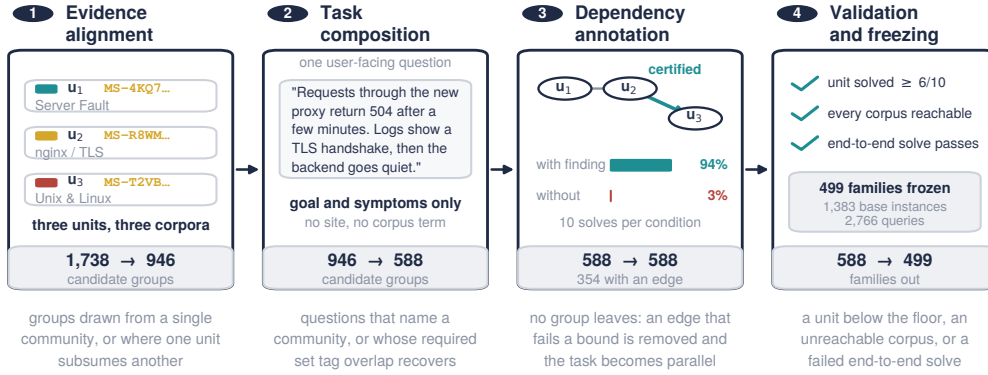


Figure 7: The task construction pipeline. A candidate group of accepted answers becomes information units, the units become one user-facing question, the dependencies among them are certified by a solve-based test, and the task is frozen once an end-to-end solve passes. The tinted box in each panel carries the candidate groups entering and leaving the stage, and the line beneath gives the rule that discards a candidate. Certification discards no group, since a group whose certified graph is empty continues as a parallel task.

checklist asks that no community be named, that only the goal and observable symptoms appear, that no term occur in a single corpus, and that every required unit be demanded without being named. A question is also rejected when its surface gives the required expertise away, which a tag-overlap baseline tests. Each of the 51 public profiles is scored by the overlap between its topic tags and the tokens of the question, and the question is rejected when the top $|R_q|$ profiles under that score already contain the full required set. Of the 358 groups discarded at this stage, 211 fail the checklist and 147 have their required set recovered by the baseline.

B.2 CERTIFYING A TASK’S DEPENDENCY GRAPH

A unit is dependent when its inputs or its objective can be fixed only once a predecessor finding is in hand. Solving settles which units those are, and it does so exhaustively. Inside a base instance every other required unit is a candidate predecessor of every unit, and what the test leaves standing is the whole graph the task carries.

Certification protocol. For a unit u_j we supply the findings of all its candidates and solve u_j against its own corpus, ten samples, then withhold one candidate at a time and solve again, ten samples each. A candidate u_i is certified as a predecessor when the full set reaches at least 8/10 and withholding u_i alone leaves u_j at or below 1/10. The two bounds do different work. The upper bound confirms that u_j is within reach once the findings it needs are present, and the lower bound confirms that this one candidate is what puts it there. Because the other candidates stay supplied throughout, the test returns immediate predecessors only. With the finding of u_i in hand, withholding something that u_i itself depends on changes nothing for u_j . The certified in-edges of every unit are the task’s dependency graph G_q , acyclic because a unit cannot be certified against a candidate it already fixes, and a base instance whose certified graph comes out empty continues as a parallel task. Of 11,962 candidate pairs, 10,130 turn out not to be necessities and 1,832 are certified, at 93.9% with the full set against 3.1% with one member withheld. Table 6 reports the certified gap for each collaboration structure. A downstream unit in a larger task arrives with more of its context already fixed by the other units, so the gap narrows as the required expertise grows, from 92.5 points at S1 in C2 to 89.7 at S3 in C3.

What a task carries. The 815 dependent base instances hold 1,832 certified dependencies between them, 2.25 each, rising with the work a task requires from 1.30 at S1 in C2 to 4.10 at S3 in C3, and a graph runs one to five edges deep with a median of two. Both phrasings of a base instance inherit its graph, so each of the 1,630 dependent queries of § 3.3 carries its own G_q and is scored on every edge of it. Certification also shapes the released set. Of the 588 groups that reach it, 354 leave with a non-empty graph and the other 234 continue to validation as parallel tasks.

Table 6: Dependency certification. We report the solve-based test that fixes each task’s dependency graph. Inside a base instance every other required unit is a candidate predecessor of every unit, and a candidate is certified when the unit clears 8/10 with the whole candidate set supplied and falls to 1/10 or below once that one candidate is withheld. The bar carries a cell’s certified edges against the largest cell, and solve rates are means over certified edges, whose gap is the necessity the test establishes. *Per task* is the certified edges of a base instance, both of whose queries inherit them.

<i>C</i>	<i>S</i>	Certified dependencies			Unit solved (%)		
		Candidates	Certified	Per task	Without	With	Gap
■ C2	S1	744	257	1.30 	2.4	94.9	92.5
■ C2	S2	1,932	355	2.26 	2.9	93.7	90.9
■ C2	S3	4,414	457	3.87 	3.2	93.5	90.2
■ C3	S1	560	194	1.31 	2.5	94.8	92.4
■ C3	S2	1,440	253	2.14 	3.5	93.9	90.4
■ C3	S3	2,872	316	4.10 	3.7	93.4	89.7
Total		11,962	1,832	2.25	3.1	93.9	90.8

Expert discovery. When the finding of u_i is also what identifies the community holding u_j , we record that edge as an expert-discovery relationship, and a task whose graph carries at least one of them enters C3. A two-sided check on the question alone certifies it. Neither the tag-overlap baseline above nor a solver that reads q together with all 51 public profiles may name the downstream community, and the same solver must name it once the finding of u_i is supplied. The first half keeps the community out of the question, the second places the clue inside the finding, so a method reaches the downstream expert through u_i . Of the 300 dependent families, 126 pass this check and form C3, and the other 174 form C2.

B.3 QUALITY CONTROL AND FAMILY STRUCTURE

Local solvability. Every required unit of every base instance is answered against its own corpus alone, ten samples, with no other agent present, covering 5,509 unit occurrences, one per unit and base instance. A unit solved in fewer than 6/10 samples is revised or dropped. Across the released set the median unit is solved in 9/10 samples and the fifth percentile in 7/10, a mean of 91.0%, which fixes an agent-side floor for everything measured downstream. The local solve stage of § 3.3 runs from 86.9% to 93.1% across the twelve methods with a mean of 90.0%, read per query over every edge of that query’s graph and only where the assignment arrived with the predecessor evidence intact. A dependency lost after that stage is lost by the team, with the agent holding it already shown capable of producing it alone.

Families. Tasks generated from the same candidate group form a family. All members of a family share the same units, the same required experts and the same collaboration structure, and differ in which observable symptoms the question surfaces and how they are worded. A family holds two or three base instances, 385 of them three and 114 two, and each base instance ships in two equivalent phrasings, so a family contributes four or six queries. All 2,766 queries are scored, and the methods of § 4 run in their native configuration without tuning on the benchmark. The family label ships with every query, because members of a family share evidence. Work that tunes on part of the set should hold out whole families, so that no variant of a scored task is seen.

Yield. The pipeline freezes 499 families from 1,738 candidate groups, 199 of them parallel and 300 dependent, with the attrition concentrated in the first two gates.

B.4 DATASET COMPOSITION

Table 7 reports the released set over the nine cells of collaboration structure and required expertise. Every cell carries at least 154 queries, enough to read a per-cell success rate at the precision reported in § 4, and the six dependency cells together hold 1,630 queries, the population over which the three pipeline capabilities of § 3.3 are measured, carrying 3,664 preregistered dependencies between them.

Table 7: Composition of the released set. We report the 2,766 queries over collaboration structure C (■ **C1** parallel, ■ **C2** sequential, ■ **C3** expert discovery) crossed with required expertise S . The bar carries the query count of a cell against the largest cell, and each base instance ships in two equivalent phrasings, so queries are twice the base instances. *Experts* gives the expert counts a cell realizes and *Mean* their average per query, which holds across the three structures at every scale to within the twelve five-expert queries of C1S2, so C and S can be read independently. The six C2 and C3 cells carry a preregistered dependency graph per query, and the 1,630 queries they hold are the population over which task orchestration, information transfer and result integration are measured.

C	S	Released set			Required expertise		
		Queries	Families	Base instances	Experts	Mean	Units
■ C1	S1	454 	82	227	2–3	2.40	1,090
■ C1	S2	390 	67	195	4–5	4.03	1,572
■ C1	S3	292 	50	146	6–8	6.60	1,927
■ C2	S1	394 	73	197	2–3	2.40	946
■ C2	S2	314 	56	157	4	4.00	1,256
■ C2	S3	236 	45	118	6–8	6.60	1,557
■ C3	S1	296 	53	148	2–3	2.40	710
■ C3	S2	236 	42	118	4	4.00	944
■ C3	S3	154 	31	77	6–8	6.60	1,016
Total		2,766	499	1,383	2–8	3.98	11,018

Within a scale bin the exact expert count follows whichever candidate groups survived certification. S2 is realized as four required experts throughout C2 and C3, C1 adds twelve five-expert queries, and S3 spreads over six, seven and eight. The per-cell counts of Table 7 are the denominators used throughout § 4.

C EVALUATION PROTOCOL

An evaluation decision in MAScope is a match between a logged trajectory and the reference of the task that produced it. Source identifiers and preregistered terms carry the match, and every verdict is stored with the trajectory, so replaying a stored log reproduces every number in the paper. This section gives what a method receives and what the environment records (§ C.1), the three outcome metrics (§ C.2), the two expert-selection metrics (§ C.3), and the five-stage decision along a query’s preregistered dependency graph (§ C.4).

C.1 EXECUTION AND LOGGING

What a method controls. A method receives the question q , the 51 public profiles, the shared agent pool, the retrieval interface of § A.2, and a budget. The question arrives with one line of answer format appended, the same for every method and every query (§ H), asking that the final answer state the identifier of every passage it rests on. The line says what an answer has to carry and nothing about how to produce it. Everything after that belongs to the method. It decides whom to recruit, how to divide the work, what each agent is told, how many rounds to run, and when to submit. MAScope supplies no coordinator, no communication topology and no termination rule of its own, so what the evaluation observes is the method’s own mechanism.

Budget and runs. Each task is capped at 2M tokens and 256 model calls, and the consumption we report is the one actually incurred. A run that reaches either cap submits whatever answer it holds at that point, and its trajectory is scored as it stands. Every model call in the benchmark decodes at temperature 0.3 with top- p 0.95 and at most 4,096 output tokens, and the construction-time solves of § B run under the same settings, so the local-solvability floor of § B.3 and the local solve rate of § C.4 sit on one decoding regime. Every method runs five times over all 2,766 queries, with the sampling seed of the backbone as the only difference between them. Queries, references, preregistered dependencies, profiles, corpora and budget are identical across the five, so the spread

Table 8: The common event stream. One adapter per method rewrites its native log into these nine events, changing no behavior and adding no field the framework did not already produce. An event a method has no counterpart for is an event its adapter never emits, and every predicate of Table 9 is decided from these fields alone.

Event	Fields	Written when
recruit	agent, round	a method adds an agent to the team for this query
assign	from, to, task text, carried units	a message delegates work to a named agent
message	from, to, content, cited identifiers	any other agent-to-agent or coordinator message
retrieve	agent, query text, returned identifiers	an agent queries its own corpus
local_output	agent, content, cited identifiers	an agent returns a result to the team
aggregate	actor, inputs, output	a method merges results into a single artifact
memory	module, operation, items	a memory module is read or written (§ 4.5)
submit	answer, cited identifiers	the method submits its final answer, or a cap is reached
usage	call, prompt tokens, completion tokens	every model call and every retrieval call

they show is decoding variance together with the scheduling nondeterminism it induces inside a method. The standard deviations in Table 1 and Table 2 are taken over those five runs.

The event stream. Twelve frameworks record their work in twelve formats, and the evaluator reads none of them directly. An adapter per method rewrites the native log into the common event stream of Table 8, changing no behavior and adding no field the framework did not already produce. An event a framework never performs is an event the adapter never emits, and a predicate that needs it is recorded as unreached. A native construct that packages several events into one is unpacked into them. A group-addressed message becomes one `assign` per recipient the reference requires, and a coordinator turn that merges results and delegates the next step becomes an `aggregate` followed by an `assign`. MoA issues no delegation of its own, and every `assign` in its trajectories comes from these two rules, since each aggregating layer merges the answers below it and hands the merged state to every agent of the next. These assignments follow the finding, but each contains the accumulated state of a layer, written for no particular unit. Table 9 asks that an assignment’s task text carry the objective terms of u_j , and a merged state does so only where it already says what is left to settle for that unit. MoA’s 34.8% in Table 2 is the share of queries reaching $s^{(1)}$ on which this holds for every edge of the graph, against more than 70% for every other method. Figure 3 shows one such stream on a service-diagnosis task, with the checks of this section marked where they apply.

C.2 OUTCOME VERIFICATION

Task success. Acceptance is decided by V , which lists for every required unit the source identifier the final answer must carry and the preregistered terms its content must state, together with the equivalent evidence and the alternative routes recorded during construction. A run succeeds when the answer satisfies every entry of V . Two kinds of match decide an entry, and both are decided by string, so acceptance is a function of the submitted answer and the reference alone. An identifier matches as an exact string, the fifteen characters of § A.3, anywhere in the submitted answer. No paraphrase produces one, which is what ties acceptance to retrieval. A preregistered term matches when the answer states it verbatim or in one of the surface variants recorded with the term during construction, which cover case, inflection, and the unit-bearing forms of a quantity. The variants are written when the task is built, frozen with it, and shipped with the reference. Nothing else is consulted. No model reads an answer to decide whether it succeeded, so the same answer scores the same way on every machine and at every later date, and every verdict comes with the exact string that produced it. A correct answer phrased outside the recorded variants is scored as a miss, which lowers every rate it touches. Writing $V_q(y_q) \in \{0, 1\}$ for the verdict on the submitted answer y_q of query q , and with $b(q, u) = 1$ when unit u appears in that answer with its identifier and its preregistered terms,

$$\text{Success} = \frac{1}{N} \sum_q V_q(y_q), \quad \text{Evidence coverage} = \frac{1}{N} \sum_q \frac{\sum_{u \in U_q} b(q, u)}{|U_q|}.$$

Table 9: Stage decisions along a query’s preregistered dependency graph G_q . Each predicate is evaluated on every edge $u_i \rightarrow u_j$ of G_q , and only where the predicate above it holds, so a query is at a stage when all of its edges are, every rate r_k is a ratio of counts over the queries that reached the previous stage, and $s^{(1)}$ is the entry condition that gives r_2 its denominator. Local solve is an agent-side reference, and it separates an agent failing to solve from a team failing to deliver.

	Capability	Rate	Predicate	Matched against
$s^{(1)}$	Ready	–	the method holds the finding of u_i and has recruited the expert holding u_j	a <code>local_output</code> citing u_i , and a <code>recruit</code> for that expert
$s^{(2)}$	Task orchestration	r_2	the finding is turned into work for that expert	an <code>assign</code> to that expert, later than the finding, whose task text carries the objective terms of u_j
$s^{(3)}$	Information transfer	r_3	the assignment carries the predecessor evidence and its key constraints	the identifier of u_i and its constraint terms present in the carried units of that <code>assign</code>
$s^{(4)}$	Local solve	r_4	the expert produces a correct result for u_j	a <code>local_output</code> from that expert citing u_j with its preregistered terms
$s^{(5)}$	Result integration	r_5	that result reaches the final answer	the identifier and terms of u_j present in the <code>submit</code> answer

Success is the conjunction of b over U_q , so evidence coverage reads the same predicate one unit at a time and records what a failed run assembled. Table 1 reports success and Table 2 reports coverage, and the distance between them is the work a method completed but never delivered.

Execution cost. Cost sums the tokens of every model call a method makes, across planning, execution, aggregation and memory, together with the retrieval calls charged to the task. Table 1 reports the per-task mean in thousands of tokens and the model calls over the five runs.

C.3 EXPERT SELECTION

Let R_q be the experts a query requires and A_q the distinct agents a method recruits. An agent the reference records as holding equivalent expertise counts toward R_q when the method recruits it in place of the primary expert. Over all 2,766 queries,

$$\text{Recall} = \frac{1}{N} \sum_q \frac{|A_q \cap R_q|}{|R_q|}, \quad \text{Precision} = \frac{1}{N} \sum_q \frac{|A_q \cap R_q|}{|A_q|}.$$

Recall on its own rewards recruiting the pool. A method that calls all 51 agents reaches a recall of 1 at a precision of 7.8%, the mean of $|R_q|/51$, which is where MoA and SelfOrg sit in Table 2. The two are therefore reported together, and neither enters the conditional rates below.

C.4 STAGE DECISIONS ALONG THE DEPENDENCY CHAIN

For every C2 and C3 query the whole dependency graph G_q is preregistered before any method runs (§ B.2), and all twelve methods are scored on the same graph of the same query. Table 9 gives the five predicates, each evaluated on an edge $u_i \rightarrow u_j$ of G_q , and what each one is matched against.

A query reaches a stage when every edge of its graph reaches it, so $s_q^{(k)} \in \{0, 1\}$ records the whole task and not a representative of it. The four rates of § 3.3 and the dependency survival of Table 2 are

$$r_k = \frac{\sum_q s_q^{(k)}}{\sum_q s_q^{(k-1)}}, \quad k = 2, \dots, 5, \quad \text{and} \quad \prod_{k=2}^5 r_k = \frac{\sum_q s_q^{(5)}}{\sum_q s_q^{(1)}}.$$

Ratios of counts weight a cell by the queries it contributes, so the rates of a method aggregate over the nine $C \times S$ cells in the proportions of Table 7. Two rewritings of § C.1 settle most of the ambiguity before it reaches a predicate, in that a group-addressed assignment becomes one assignment per required recipient, and a local output citing no identifier is matched on its preregistered terms alone.

An edge that survives both undecided is labeled so, and a query holding one leaves the numerator and the denominator of its rate together, so the rate is read over the queries whose graphs the trajectory settles in full. The label travels with the trajectory, so a rate can be recounted on whatever population a reader chooses.

D COMPARED METHODS

A method enters MAScope through one interface. It reads the 51 public profiles, calls agents through the retrieval-backed interface of § A.2, and submits a final answer. Everything between those two points is the method’s own code running the mechanism its authors published. This section gives how the twelve methods are integrated and what each decides for itself (§ D.1), the single agent reported alongside them (§ D.2), the stricter stopping rule of § 4.4 (§ D.3), and the three memory modules of § 4.5 (§ D.4).

D.1 INTEGRATION

What the integration replaces. Every framework ships with its own notion of an agent, usually a role, a system prompt and a set of tools. The integration replaces that notion with a MAScope expert. The role becomes the public profile, and the tool set becomes the single retrieval call into that expert’s corpus. The coordinator, the message routing, the planning loop and the stopping rule run as published, so a difference between two rows of Table 2 is a difference between two mechanisms. Each method runs from its authors’ public release, and the revision we ran ships with our code together with the adapter that rewrites its log into the event stream of § C.1. One framework carries state of its own across queries. AgentNet builds a routing memory as it works, and the main experiments run with that memory switched off, so all twelve rows of Table 1 stand on within-query behavior and the modules of § D.4 are the only place cross-task state enters.

What a method decides. All twelve see the same pool and the same profiles, and each applies its own rule to them. Table 10 gives that rule alongside the condition under which a run ends. The rules differ most in how many agents they admit. A framework whose team size is a parameter of the mechanism admits the whole pool, which is why MoA and SelfOrg sit at a recall of 1 and a precision of 7.8% in Table 2, while a framework that staffs from the profiles admits a handful and trades recall for precision. Where a mechanism fixes a structure we keep the one its paper specifies. MoA runs three layers, the first proposing and the rest aggregating. MAD runs three debate rounds before the judge rules, over a panel of two or three debaters, which gives the mean team of 3.7 that includes the judge. AgentVerse runs at most three recruit, decide, act and evaluate rounds. CrewAI and LangGraph ship several coordination patterns, and each runs the one built for delegation. CrewAI uses its hierarchical process, in which a manager assigns each task of a list fixed at kickoff to the agent it chooses, and LangGraph uses its supervisor pattern (Barda, 2025), in which one supervisor routes each step to a single worker. Every other round, hop and retry limit keeps the framework’s published default, and a run that reaches a limit submits what it holds, exactly as a run that reaches the budget does (§ C.1).

Tools and the backbone. Every method and every agent runs on `claude-sonnet-5` (Anthropic, 2026) under the decoding settings of § C.1. Retrieval into a corpus is the only tool the environment exposes, so a framework that ships with web search, code execution or file handling runs with those tools removed and its prompts adjusted to the one call that remains. Every agent receives that call under the same declaration, a natural-language query over its own corpus returning at most eight passages with their source identifiers. Where a framework’s prompt names a tool the environment does not expose, the sentence naming it is deleted, so every prompt a framework runs is a subset of the prompt it ships with. The answer format of § C.1 travels with the question, so it adds nothing to any prompt.

D.2 THE SINGLE AGENT REFERENCE

The reference removes the division of labor altogether. One agent receives the question and the 51 public profiles, reaches every corpus through the retrieval interface of § A.2, and writes the answer itself, so no finding ever has to pass from one agent to another. It is the expert of no community and

Table 10: The twelve methods and the decisions each one owns. Every mechanism runs as published, and the integration replaces only the notion of an agent. *Team* is the measured mean number of distinct agents a method recruits per query, so a staffing rule can be read against the team it produces. The reference agent recruits nobody and holds all 51 corpora in one context.

Method	Team	Staffing rule	How the work is organized	A run ends when
Centralized				
AutoGen	4.4	staffed from the profiles	coordinator picks the next speaker each turn	the coordinator emits its termination message
Magentic-One	4.5	staffed from the profiles	task and progress ledgers, re-planned on a stall	the progress ledger marks the request satisfied
Flow	4.6	staffed from the profiles	explicit subtask graph, updated during execution	every node of the graph is complete
CrewAI	4.4	fixed crew, staffed before the run	manager delegates each planned task once	the last planned task returns
LangGraph	4.5	staffed from the profiles	supervisor routes to one worker per step	the graph reaches its end node
Semi-centralized				
Anemoui	4.5	staffed by the planner	agents talk directly in a channel the planner opens	the planner judges the plan complete
AgentVerse	5.2	recruiter staffs a team each round	recruit, decide, act, evaluate, then repeat	the evaluator declines another round
MoA	51.0	the whole pool proposes	three layers that propose then aggregate	the last layer returns
MAD	3.7	a debater panel plus one judge	fixed debate rounds between the debaters	the judge returns its verdict
Decentralized				
GoA	4.8	staffed from the profiles	agents propagate partial results over a graph	no message changes the shared state
AgentNet	3.1	agents route to one another	routing by a capability index over the profiles	an agent returns a final answer
SelfOrg	51.0	the whole pool is available	agents self-select onto open work	no agent claims the work that remains
Reference				
Single Agent	–	no recruitment	one execution context with retrieval into any corpus	the agent writes its answer

runs a prompt of its own (§ H) in a plain tool loop, where each model call either retrieves from a corpus it names or writes the final answer, with no planner or reflection step added. Deciding where to look is its problem as much as a team’s, and every passage it retrieves stays in the one context that writes, so what a team has to carry between agents it already holds. Its question carries the same answer format as every method’s (§ C.1). It runs under the same backbone, decoding and caps as every method and uses a mean of 11.9 calls and 131k tokens per task, far inside both caps. Its 32.6% in Table 1 is what one context holding the whole task delivers, and § F.4 reads the teams against it.

D.3 EXHAUSTIVE COLLABORATION

§ 4.4 replaces each method’s stopping rule with a stricter one. A run continues until the method confirms that every requirement of q is met and that no recruited agent can add to the answer. The confirmation is made by the method itself, at the point where its own mechanism already decides whether to continue. A coordinator makes it where it would otherwise emit its termination message, a planner or an evaluator where it would otherwise declare the plan complete, a judge in place of its verdict, and a decentralized method locally in each agent, with the run ending once the last agent reports nothing to add. The judgment therefore stays inside the method, and what a run confirms is the method’s own notion of completeness. Everything else is unchanged, so a method keeps planning, routing and merging as always and only stops later. Tokens spent in the extra rounds are charged to the task, the budget of § C.1 still binds, and Figure 5 reports the multiplier.

D.4 MEMORY MODULES

Three modules attach to a host method at three different points. A-MEM (Xu et al., 2025) keeps cross-task experience as structured notes on each agent with links between them, written when a task closes and read when that agent is called again. Blackboard (Han & Zhang, 2025) holds shared working state that the team reads and writes inside one task, and it starts empty on every query. G-Memory (Zhang et al., 2025a) keeps a three-level graph of experience, tasks and interactions, and retrieves from it across tasks and across the steps of the current one.

Two of the three accumulate across tasks, so what a module holds at a given query is a function of the queries that came before it. That accumulation happens on the evaluation stream itself. Every module starts empty and grows only from the trajectories its host produces while being measured, which is the behavior the module exists to provide. Every query of MAScope is scored (§ B.3), so a store built in advance would have to come from the evaluation set, and a store frozen before evaluation and restored per query would leave A-MEM and G-Memory working as static indexes.

Accumulation makes the order of the queries part of the experimental condition, and two rules fix it. The order is drawn once and frozen, and the same order runs for every host, every module and all five runs, so a query meets the same amount of accumulated experience whichever combination is being measured. An entry written while solving one query also stays out of reach of the other queries of its family. Members of a family share units and therefore share identifiers (§ B.3), and holding them apart keeps a sibling’s answer out of the store, so the identifier acceptance turns on still arrives through retrieval. Blackboard needs neither rule, since it lives and dies inside one query.

With nothing populated in advance, every token the modules cost is spent inside a scored query. Reads, writes, the retrieval a module performs over its own store, and the extra context it adds to a call are charged to the query where they happen, which is what Figure 5 reports as a module’s cost.

E PRELIMINARY STUDY

§ 2 asks whether the final outcome of a MAS reflects the collaboration behind it. We ask it on two existing benchmarks, MuSiQue (Trivedi et al., 2022) for multi-hop question answering and GAIA (Mialon et al., 2024) for general assistant work, and we read the answer out of the trajectories they produce under their own mechanisms. This section gives the tasks and settings (§ E.1), the masking protocol behind the first observation (§ E.2), and the rule behind the second (§ E.3).

E.1 TASKS AND SETTINGS

Tasks come from the published splits of both benchmarks, 200 four-hop questions sampled from the MuSiQue-Ans development split and all 165 tasks of the GAIA validation split. The two benchmarks differ in where evidence comes from. MuSiQue evidence is textual and sits in passages an agent reads, while GAIA evidence comes from file processing and web retrieval, so it enters a trajectory only when an agent performs the operation that produces it. That contrast is what the first observation turns on.

Three representative methods run on both, AutoGen (Wu et al., 2024), CrewAI (Moura, 2023) and Anemol (Ren et al., 2025). Each runs with four agents and a budget of 150k tokens per task, on one shared backbone, `claude-sonnet-5` (Anthropic, 2026) at temperature 0.3 with top- p 0.95, the settings of § C.1. The tool set is the one each benchmark presumes. On MuSiQue the agents read the paragraph pool the question ships with and nothing else, so every finding is a passage one of them retrieved. On GAIA they hold the task’s attached file, a web search tool and a Python interpreter, which is the standard GAIA setup and the reason a finding there exists only once an agent has run the operation that produces it. Every configuration runs five times, and Figure 2 reports the mean.

E.2 MASKING A REQUIRED CONTRIBUTION

A run is scored twice from the same trajectory. We freeze every agent record at the point before aggregation, then re-aggregate under the method’s own rule in two conditions. Condition (I) reads the complete record. An agent counts as required when it holds a step the task cannot be answered without, and both benchmarks state those steps themselves. MuSiQue ships the supporting para-

Table 11: The paired counts behind Figure 2. We report both aggregation conditions on the same frozen trajectory, so every query carries two verdicts, with n of 200 on MuSiQue and 165 on GAIA and every value a mean over five runs. *Only I* counts the queries the complete record answers and the masked record misses, *Only II* counts the reverse, and their difference over n is the drop that Δ states as a share of full-record accuracy, taken over methods in the averaged rows. *Lost* counts the failures whose record already held a required finding the answer never adopted, and *Share* takes it over pooled failures.

Method	Aggregation conditions					Findings lost		
	Full	Masked	Only I	Only II	Δ	Failures	Lost	Share
<i>MuSiQue</i>								
AutoGen	64.3	60.8	9.2	2.2	-5.4	71.4	18.6	26.1
CrewAI	62.5	51.2	25.0	2.4	-18.1	75.0	22.2	29.6
Anemoui	58.2	50.1	20.0	3.8	-13.9	83.6	27.8	33.3
Avg.	61.7	54.0	18.1	2.8	-12.5	76.7	22.9	29.8
<i>GAIA</i>								
AutoGen	41.7	26.8	28.0	3.4	-35.8	96.2	38.6	40.1
CrewAI	39.5	17.1	42.2	5.2	-56.7	99.8	43.8	43.9
Anemoui	33.7	15.6	33.8	4.0	-53.6	109.4	56.6	51.7
Avg.	38.3	19.8	34.7	4.2	-48.7	101.8	46.3	45.5

graphs of every hop, and GAIA ships the annotator’s own solution steps together with the tools each one took, so in both cases the agent that covers a listed step is required. Condition (II) reads the same record with the evidence of one required agent removed, and which agent that is comes from the query hash before any run starts, so all three methods lose the same contribution on the same task. Removal covers the passages that agent retrieved, the paraphrases of them that appear in its messages, and any direct statement of its finding inside a summary. A span goes when it states one of the masked agent’s required findings, wherever it sits. Two of the authors mark the spans on the frozen record before either condition is scored, and a third resolves disagreements. The agent’s participation, its other messages and every other agent’s record stay as they were, so what condition (II) takes away is one contribution.

Re-aggregation is a model call, so it is not deterministic, and a small number of queries come out correct under (II) after failing under (I). Table 11 reports those queries as *Only II*, between 2.2 and 5.2 per configuration, against the 9.2 to 42.2 of *Only I* that go the other way. The drop we report is the net difference between the two conditions, so both directions enter it.

The distance between the two conditions is what the team loses by losing one required contribution. Averaged over the three methods and five runs, it is 12.5% on MuSiQue and 48.7% on GAIA (Figure 2, left). Per method the relative drop is 5.4%, 18.1% and 13.9% on MuSiQue for AutoGen, CrewAI and Anemoui, which is 3.5, 11.3 and 8.1 points of accuracy, and 35.8%, 56.7% and 53.6% on GAIA, which is 14.9, 22.4 and 18.1 points. The ordering is the same on both benchmarks and follows how much of the record reaches the aggregator, which is a different ordering from final accuracy. AutoGen carries the whole transcript and loses least, while CrewAI’s agents deliver to a manager and never see each other, and it loses most.

E.3 LOCATING A DISCARDED FINDING

The second observation reads only the runs that failed under condition (I) and asks whether the answer they needed was already in the record. A failed run holds the decisive finding when some agent’s output states the content the reference requires for the step the run got wrong. The match is decided against the reference’s own statement of that step, and a finding counts when it supplies what the step needs, in whatever words it is phrased. The same two authors mark it independently and the third resolves disagreements, as for the masking spans. The check reads the frozen record on its own, so a finding counts whether or not the aggregator ever referred to it.

Among the failed runs, 29.8% on MuSiQue and 45.5% on GAIA hold the decisive finding (Figure 2, right). Per method the share is 26.1%, 29.6% and 33.3% on MuSiQue for AutoGen, CrewAI and Anemoui, and 40.1%, 43.9% and 51.7% on GAIA. This ordering differs from the one above. It fol-

lows how a method delivers what its agents produce, and it runs opposite to final accuracy. Anemoui summarizes a free discussion into an answer, and it both scores lowest and discards the most, while AutoGen scores highest and discards the least. The two observations separate two properties of the same record, and neither is visible in the outcome a benchmark reports.

F FULL RESULTS

The main text reports rates, and this section reports the counts they come from. § F.1 follows the 1,630 dependent queries stage by stage, each carrying its whole preregistered dependency graph, so every rate of Table 2 can be recomputed from adjacent columns. § F.2 and § F.3 give the two enhancement experiments in full, with the success and cost that Figure 5 plots as differences, and § F.4 reads the twelve methods against the single agent, the one system in the paper with no handoff at all.

F.1 WHERE DEPENDENT QUERIES GO

Table 12 gives the population at each stage, counted over queries, and a query is at a stage when every edge of its preregistered graph has reached it. One column stands before $s^{(1)}$ and one behind $s^{(5)}$. The first counts the queries whose every predecessor finding the team holds at all, the retrieval a method has to complete before any of the four capabilities applies. $s^{(1)}$ adds that every downstream expert has been recruited, and it is the denominator the first rate is read against. The last counts the dependent queries the method answers, which sits below $s^{(5)}$ because carrying a graph into the answer leaves the rest of the query to be satisfied.

Reading down a column shows where each class thins out. MoA enters the chain on 1,408.6 of the 1,630 queries, the highest of the twelve and equal to the count that holds the findings, because recruiting the whole pool means no downstream expert is ever missing. It keeps 490.4 of them after orchestration. AgentNet enters on 788.0, the lowest of the twelve, and keeps 690.2, so the two arrive at a similar place from opposite ends, one by recruiting everyone and then failing to dispatch, the other by recruiting few and dispatching what it has. The rates of Table 2 report the second half of each of those stories, and these counts report the first.

F.2 EXHAUSTIVE COLLABORATION

Table 13 gives the native and exhaustive success of every method alongside the token ratio between the two rules. The ratio runs from 1.29 to 3.34, and the three cheapest continuations are the three methods whose plan is fixed before execution, since another round re-executes what that plan already holds. The success the extra rounds buy runs from -0.77 to $+1.63$ points. All seven methods that re-plan when a finding arrives land on the positive side, and the four that lose ground, CrewAI, LangGraph, AgentVerse and MAD, either fix their plan before execution or merge by adjudication.

The run-to-run spread says which of these effects hold, since a sign alone does not separate an effect from noise. Nine of the twelve clear twice their own spread, and they divide without exception. Six are gains, every one from a re-planning method: Flow, AutoGen, Magentic-One, AgentNet, SelfOrg and GoA. Three are losses, every one from a mechanism that cannot act on a finding at all: CrewAI with its plan fixed before execution, and MAD and AgentVerse with their judges. The three that stay inside two spreads are LangGraph and MoA, whose plans are fixed and whose change is therefore expected to be nothing, and Anemoui. Anemoui is the one re-planner we cannot distinguish from zero, and Table 2 says why. Its orchestration is already the highest of the twelve at 92.0%, and its integration the second lowest at 76.4%. The extra rounds do produce further findings for it, and the merge then discards them.

F.3 MEMORY MODULES

Table 14 gives all twelve host and module pairings with the success and the cost of each. The host row repeats in every block, so a pairing can be read against its own baseline and against the other two modules on the same host. Cost moves in a narrow band, between -4.5% and $+11.9\%$, while success moves between -1.13 and $+3.22$ points, so the whole spread in success comes from which

Table 12: Where the 1,630 dependent queries go along their preregistered graphs. Counts are means over five runs, and a query is at a stage when every edge of its graph has reached it. *Finding held* counts the queries whose every predecessor finding the team obtained at all, which is the retrieval that has to succeed before any capability applies, and $s^{(1)}$ adds that every downstream expert was recruited to give r_2 its denominator. Each rate of Table 2 is the ratio of two adjacent columns. *Answered* counts the dependent queries the method solves, and sits below $s^{(5)}$ because carrying a whole graph into the answer still leaves the rest of the query to satisfy. The stage populations vary between 2.5 and 38.6 across the five runs, the spread of the rates they produce is in Table 2, and the spread of the outcome is given here.

Method	Before	Stage populations					Outcome
	Finding held	$s^{(1)}$	$s^{(2)}$	$s^{(3)}$	$s^{(4)}$	$s^{(5)}$	Answered
 <i>Centralized</i>							
AutoGen	1,310.4	1,172.4	1,002.2	900.2	824.8	761.4	691.0 $\pm$ 8.7
Magentic-One	1,296.2	1,157.6	1,001.4	836.4	763.4	703.8	652.2 $\pm$ 14.4
Flow	1,219.0	1,087.6	904.4	772.4	698.0	629.2	536.2 $\pm$ 4.0
CrewAI	1,189.6	1,069.8	758.4	620.2	554.2	509.2	460.4 $\pm$ 12.8
LangGraph	1,280.8	1,097.0	847.6	722.8	648.4	612.4	543.2 $\pm$ 10.0
 <i>Semi-centralized</i>							
Anemoui	1,346.4	1,177.4	1,083.0	872.0	811.4	619.8	550.2 $\pm$ 13.1
AgentVerse	1,335.8	1,197.4	910.2	701.0	634.8	506.6	475.8 $\pm$ 22.0
MoA	1,408.6	1,408.6	490.4	380.8	344.4	275.4	260.2 $\pm$ 3.7
MAD	1,022.2	832.2	730.4	608.6	533.0	394.0	321.4 $\pm$ 20.3
 <i>Decentralized</i>							
GoA	1,238.4	1,134.2	1,040.6	746.4	659.6	575.8	556.8 $\pm$ 9.7
AgentNet	957.0	788.0	690.2	547.4	495.2	425.4	421.2 $\pm$ 15.9
SelfOrg	1,059.0	1,059.0	905.2	593.8	515.8	419.0	368.4 $\pm$ 16.0

Table 13: Exhaustive collaboration in full. We report each method under its native protocol and under the requirement to collaborate until it confirms completion, as means over five runs on all 2,766 queries, with cost in thousands of tokens per task. Δ is the difference in success rate, and the two conditions share a seed within a run, so its spread is that of the paired difference. *Mechanism* is how a method handles a finding produced at run time, the grouping Figure 5 plots by, and the token ratio is the exhaustive cost over the native one.

		Successes			Cost (k tokens)		
Method	Mechanism	Native	Exhaustive	Δ	Native	Exhaustive	$\times$
 <i>Centralized</i>							
AutoGen	■ Re-plans	1,442.2	1,487.4	1.63$\pm$0.40	170.9	482.3	2.82
Magentic-One	■ Re-plans	1,388.4	1,420.2	1.15$\pm$0.31	208.4	357.7	1.72
Flow	■ Re-plans	1,289.0	1,323.6	1.25$\pm$0.17	126.2	361.3	2.86
CrewAI	■ Plan fixed	1,241.0	1,233.6	−0.27$\pm$0.12	118.8	176.8	1.49
LangGraph	■ Plan fixed	1,169.0	1,168.4	−0.02$\pm$0.23	127.4	174.2	1.37
 <i>Semi-centralized</i>							
Anemoui	■ Re-plans	1,224.6	1,233.6	0.33$\pm$0.29	118.3	306.1	2.59
AgentVerse	■ Judge merges	1,160.0	1,143.6	−0.59$\pm$0.18	165.2	375.9	2.28
MoA	■ Plan fixed	884.2	889.2	0.18$\pm$0.18	1,044.3	1,346.7	1.29
MAD	■ Judge merges	814.4	793.2	−0.77$\pm$0.21	125.0	370.7	2.97
 <i>Decentralized</i>							
GoA	■ Re-plans	1,276.8	1,297.4	0.74$\pm$0.13	208.2	420.8	2.02
AgentNet	■ Re-plans	933.4	961.2	1.01$\pm$0.24	87.5	292.4	3.34
SelfOrg	■ Re-plans	892.2	914.0	0.79$\pm$0.26	323.6	512.1	1.58

module supplies what the host lacks, at close to the same cost throughout. The four largest gains are the four pairings that place a module at the level of the host’s weakest stage, A-MEM on AgentNet, G-Memory on AutoGen and on GoA, and Blackboard on MoA, and each of them clears its own run-to-run spread by between 7.7 and 15.1 times. The largest loss clears it by 6.5, GoA with A-MEM at -1.13, where per-agent notes attach to a method whose agents already share a graph and add context to every call without adding reach. Cost tracks none of this. A-MEM saves AgentNet

Table 14: Memory modules in full. Each host appears with its own run and with the three modules attached to it, all means over five runs on 2,766 queries. Δ is the success rate against that host’s own baseline in points, and the last column the change in per-task tokens. Both deltas carry their spread over the five runs, taken on the paired difference, since a module and its baseline share a seed within a run. The modules accumulate on the evaluation stream under the rules of § D.4.

Host	Module	Success		Δ	Cost	
		Successes	Rate		k tokens	$\Delta\%$
AutoGen	none	1,442.2	52.14	–	170.9	–
	A-MEM	1,459.6	52.77	0.63± 0.37	178.8	4.6 ± 0.9
	Blackboard	1,484.2	53.66	1.52± 0.38	175.2	2.5 ± 1.1
	G-Memory	1,524.0	55.10	2.96± 0.20	184.4	7.9 ± 1.8
GoA	none	1,276.8	46.16	–	208.2	–
	A-MEM	1,245.6	45.03	–1.13± 0.17	217.0	4.2 ± 1.0
	Blackboard	1,266.6	45.79	–0.37± 0.18	214.3	2.9 ± 0.8
	G-Memory	1,354.0	48.95	2.79± 0.36	220.5	5.9 ± 0.8
AgentNet	none	933.4	33.75	–	87.5	–
	A-MEM	1,022.4	36.96	3.22± 0.22	83.6	–4.5 ± 0.8
	Blackboard	971.4	35.12	1.37± 0.21	91.5	4.6 ± 1.1
	G-Memory	950.2	34.35	0.61± 0.24	97.9	11.9 ± 1.1
MoA	none	884.2	31.97	–	1,044.3	–
	A-MEM	898.2	32.47	0.51± 0.17	1,058.1	1.3 ± 0.9
	Blackboard	929.8	33.62	1.65± 0.14	1,076.2	3.1 ± 0.6
	G-Memory	873.4	31.58	–0.39± 0.23	1,083.0	3.7 ± 0.7

4.5% of its tokens while G-Memory costs AutoGen 7.9% and GoA 5.9%, so what a module delivers follows from the level it attaches at.

F.4 THE NO-COLLABORATION REFERENCE

The single agent of § D.2 solves every task without a handoff, which makes it the reference for what dividing the work contributes. Its success falls by 46.5% from C1 to C2 and C3, against a mean of 48.6% over the twelve methods. Both paradigms meet the same demand there, a finding carried to the unit that cannot be scoped without it, one inside a single context and the other across agents, and a dependent unit that receives its finding is solved about nine times in ten (Table 2), so what either loses is lost in the carrying.

The two paradigms part on scale. The reference falls by 45.6% from S1 to S3 against 38.2% for the mean team, and inside C1, where no finding has to travel at all, it trails the mean team by 16.1% at S1 and by 31.7% at S3. One context holding every community of a task pays more the more communities the task needs, and dividing the work across agents is what relieves it.

Read against the reference, a method’s dependency drop is a net effect, the relief that dividing the work brings minus what its handoffs drop. In Table 1 the difference runs from a drop 10.6 points smaller than the reference’s, for AutoGen, to one 24.4 points larger, for MoA, and across the twelve methods it tracks the dependency survival of Table 2, with a Pearson correlation of 0.88 and a Spearman correlation of 0.86, or 0.75 and 0.82 with MoA set aside. The methods that fall furthest behind the reference are the ones whose handoffs drop the most dependencies, which is the loss the four capabilities locate. Two limits apply. The reference is one agent that every method is read against, not a counterfactual of any one of them, and survival and dependent success are measured on the same dependent queries, so the correlation shows that the two move together and leaves the direction open.

G QUALITATIVE EXAMPLES

The three mechanisms of § 4.3 are statements about what a trajectory contains, and this section shows the trajectories. § G.1 gives one task in full, so every excerpt that follows can be read against what the reference asks for. § G.2 to § G.4 then pair a run that loses one of that task’s preregistered dependencies with a run that carries it, on the same edge, so the two boxes differ only in the mechanism that handles the finding. Excerpts are the events of Table 8 with their content abridged, and identifiers, the terms the reference preregisters and the order of events are as logged.

G.1 ONE TASK IN FULL

The example is a service-diagnosis query from C2 at scale S3, the cell of Table 7 that holds 236 queries over six to eight required experts. This one requires six. Two of them the question reaches on its own, and the other four are scoped only once the diagnosis in the first two is in hand. Certification (§ B.2) returns six edges between them, and all six are preregistered. Figure 8 gives the task in full, with the graph the runs below are scored against.

G.2 PLANNING RIGIDITY

CrewAI runs a task list fixed before execution, so a finding produced at run time reaches the manager with nowhere to go. The first box is the turn where the finding arrives and the next planned task is issued, and the work the finding implies is never issued at all. The second box is AutoGen on the same edge, where the manager reads the same finding and selects the expert it points to.

CrewAI, the finding arrives and the plan continues

local_output **Network Engineering** MS-9TJ5XB2WQNMF
 Reachability and response speed are different tests. The direct check succeeds while the proxied path stalls for 40 s against a 30 s read timeout, so the path is not the fault and the remedy has to change timing.

assign **manager** → **Database Administrators** *plan item 3 of 4*
 “Investigate the 12 s lock wait on the reporting query.” carried units: MS-4KQ7T2VB9XHD

The remaining turns issue plan item 4 and the final assembly. No assign naming the TLS remedy is issued, and the Security agent, recruited at staffing time, is never given work.

submit **manager**
 The answer carries MS-4KQ7T2VB9XHD, MS-9TJ5XB2WQNMF and MS-Z5RB8KM2XHDQ, and states a timeout increase with no verification constraint.

Stage decision. $s^{(1)}$ holds, the finding is in hand and Security is on the team. $s^{(2)}$ fails, no assignment follows the finding.

AutoGen, the same finding reaches the speaker selection

local_output **Network Engineering** MS-9TJ5XB2WQNMF
Identical content, same position in the run.

message **coordinator, speaker selection**
 “The path is clear, so the fix is timing on the proxy. Whether the budget can be raised depends on what the TLS layer requires. Next speaker: Security.”

assign **coordinator** → **Security**
 “Give a timing-only remedy for the 30 s read timeout against 40 s of upstream silence, and state what has to hold for certificate verification.” carried units: MS-4KQ7T2VB9XHD, MS-9TJ5XB2WQNMF

Stage decision. $s^{(2)}$ holds, the assignment follows the finding and carries the objective of E3.

G.3 SEMANTIC COMPRESSION

GoA moves each finding on as a round summary, and a summary carries what the round settled. The first box puts the predecessor’s full local output beside the message its successor receives, with

A C2 query at scale S3, with six required experts

The C2 / S3 cell contains 236 queries over six to eight required experts; this task requires six.

Query q

“A reporting service started timing out after we moved it behind a reverse proxy. Direct access to the service still works. The proxy logs a gateway timeout at 30 s, the upstream goes quiet for about 40 s before that, the client retries three times, and one report query waits about 12 s on a lock. Explain the cause and give a remedy that covers the proxy, the TLS layer, the retries and the database together, and that we can put into production without a second outage.”

Required units U

E1 · Server Fault [MS-4KQ7T2VB9XHD](#)

The gateway timeout is raised by the proxy, where 40 s of upstream silence exceeds a 30 s read timeout. *Preregistered terms*: proxy-side timeout, 40 s > 30 s.

E2 · Network Engineering [MS-9TJ5XB2WQNM](#)

Reachability and response speed are different tests, so a working direct path leaves the proxied one untested and the remedy is a timing change. *Preregistered terms*: reachability $\neq$ response speed, timing remedy.

E3 · Security [MS-H6PD3RK8ZVCM](#)

Certificate verification stays on, and the handshake budget is fixed by raising the read timeout. *Preregistered terms*: verification preserved, timeout raised.

E4 · Stack Overflow [MS-W2NC7FQ4JB6T](#)

Retries must be bounded so that attempts times timeout stays inside the new budget, and each retry repeats the handshake E3 keeps. *Preregistered terms*: bounded retries, retry cost includes the handshake.

E5 · Database Administrators [MS-Z5RB8KM2XHDQ](#)

The 12 s lock wait belongs to a blocking session that has to be identified, and it has to fit inside whatever budget the proxy ends up with. *Preregistered terms*: blocking session identified, 12 s inside the budget.

E6 · DevOps [MS-B7XQ4M2ZHPKT](#)

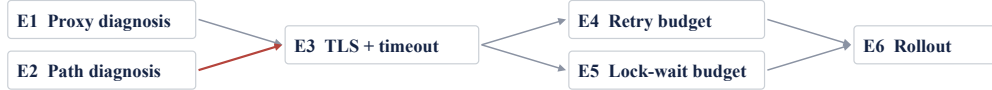
The new timeout reaches production by draining in-flight connections and reloading the proxy on one node at a time, so applying the remedy does not itself drop requests. *Preregistered terms*: drain before reload, staged rollout.

Dependency graph G_q

Reachable from the question

Scoped only once their prerequisite findings are available

Supplied $\geq 8/10$; withheld $\leq 1/10$



All six edges are preregistered and scored, so the query reaches a stage only when every edge does.

E1 and E2 establish timing on the proxy side. E3 fixes the budgets for E4 and E5, and E6 waits until that budget is final.

E2 $\rightarrow$ E3 is the edge examined by the three trajectory pairs in G.2–G.4.

E1 $\rightarrow$ E5 is not certified: with E3 supplied, withholding E1 leaves E5 solvable, so necessity runs through E3.

Acceptance V

The answer carries all six identifiers and states the preregistered terms of each unit. Two equivalences are recorded: a remedy that raises the read timeout to any value above the observed upstream silence satisfies E3, and naming the blocking session by query or by session identifier satisfies E5. No alternative route is recorded, so every unit must be retrieved from its own corpus.

Figure 8: One task in full. The query a method receives, the six information units a solution requires with the community holding each one, its source identifier and its preregistered terms, the certified dependency graph G_q , and the acceptance criteria. An arrow is a certified dependency, the unit at its head clearing 8/10 once the findings it needs are supplied and falling to 1/10 or below when the one at its tail is withheld (§ B.2). All six edges are scored, so the query reaches a stage only when its whole graph has, and the excerpts of § G.2 to § G.4 read the edge in red.

the source identifier and the constraint terms the summary leaves out marked. The second box is AutoGen on the same edge, where the transcript reaches the next agent unchanged.

GoA, the finding and the summary that carries it

local_output **Network Engineering** [MS-9TJ5XB2WQNM](#)

“Reachability and response speed are different tests, so the working direct path does not clear the proxied one. The proxied path stalls for 40 s of upstream silence against a 30 s read timeout, so any remedy has to fit inside that budget or raise it deliberately.”

aggregate **round 2 summary, broadcast to the graph**
 “Path is fine. The problem is timing on the proxy.”
 Dropped by the summary: the identifier `MS-9TJ5XB2WQNMf`, the 40 s and 30 s figures, and the constraint that a remedy must fit or raise that budget.
 assign **graph** → **Security**
 carried units: **none**. The Security agent receives the summary sentence alone.
 Stage decision. $s^{(2)}$ holds, the work is dispatched. $s^{(3)}$ fails, neither the identifier of E2 nor its constraint terms reach the expert that needs them.

AutoGen, the same finding delivered intact

assign **coordinator** → **Security**
 “Give a timing-only remedy for the 30 s read timeout against 40 s of upstream silence, and state what has to hold for certificate verification.” carried units: `MS-4KQ7T2VB9XHD`, `MS-9TJ5XB2WQNMf`
 The assignment restates both figures and the budget constraint, and the transcript of the round remains readable to the recipient.
 Stage decision. $s^{(3)}$ holds, the identifier and the constraint terms of E2 are present in the carried units.

G.4 ADJUDICATIVE AGGREGATION

MAD closes with a judge that picks among candidate answers, so a correct local result enters the final step as one candidate among several. The first box is the judge’s record on a query where the correct result was on the table and another candidate was returned. The second box is LangGraph on the same query, where the agents’ results are assembled into one answer and the correct one keeps its place.

MAD, a correct result loses the adjudication

local_output **Security** `MS-H6PD3RK8ZVCM`
 “Keep certificate verification. Raise the read timeout to 45 s, above the observed 40 s of silence, and leave the handshake budget where it is.”
 message **debater B** → **judge**
 “Candidate B: raise the read timeout to 45 s and keep verification as it is.” cites `MS-H6PD3RK8ZVCM`
 message **debater A** → **judge**
 “Candidate A: relax strict verification so the handshake finishes inside the current 30 s.” cites nothing
 aggregate **judge, round 3 verdict**
 “A. It addresses the handshake directly and changes one setting where B changes two.”
 submit **judge**
 The answer carries `MS-4KQ7T2VB9XHD` and `MS-9TJ5XB2WQNMf`. `MS-H6PD3RK8ZVCM` is **absent**, and the answer contradicts the term E3 preregisters.
 Stage decision. $s^{(4)}$ holds, the expert produced the correct result. $s^{(5)}$ fails, the result never reached the answer.

LangGraph, the same result assembled into the answer

local_output **Security** `MS-H6PD3RK8ZVCM`
 Identical content.
 aggregate **supervisor, assembly step**
 “Proxy: raise the read timeout to 45 s `MS-4KQ7T2VB9XHD`. TLS: verification unchanged `MS-H6PD3RK8ZVCM`. Retries: bounded at three within the 45 s budget `MS-W2NC7FQ4JB6T`. Database: the blocking session is identified and its 12 s wait fits `MS-Z5RB8KM2XHDQ`. Rollout: drain and reload one node at a time `MS-B7XQ4MZ2HPKT`.”
 The supervisor writes each worker’s result into its own slot, so no result competes with another for a place in the answer.
 Stage decision. $s^{(5)}$ holds, the identifier and the terms of E3 are present in the submitted answer.

The three losses sit at three different stages of the same edge. In each one the Security agent was on the team and never asked, was asked without what it needed, or answered correctly and was overruled, so the expertise was present throughout. Any one of them costs the query its whole graph, since a query reaches a stage only when every edge of Figure 8 has. That is the distinction Table 12 counts and Table 2 rates.

H PROMPTS

MAScope adds four prompts and one line of its own, and none of them scores anything. Three reach scored runs. The expert prompt is what every one of the 51 expert agents receives, where the corpus boundary and the identifier requirement of § A.2 become instructions an agent can follow. The answer format is one line appended to every question q , for every method and for the single agent alike, so whichever agent writes the final answer is told to keep the identifiers, whatever framework it runs in. The single-agent prompt runs the no-collaboration reference of § D.2, and nothing else. The other two run at construction. The expert-discovery probe of § B.2 decides which tasks enter C3, and a composer, not printed, drafts a composed question from a group of units, which the author checklist of § B.1 keeps or discards. Each method keeps its own prompts as its authors wrote them (§ D.1) and each memory module keeps its own, so these are the whole of the text MAScope adds. Acceptance itself uses no prompt, since every entry of V is settled by string against the reference (§ C.2). The answer format tells a method what an answer has to carry, and only the string match reads it.

The expert agent

You are the expert agent for <site>, one of 51 experts a method may recruit for a user’s question. Your scope: <expertise scope>.

You hold a private corpus of accepted question-and-answer pairs from <site>. It is the only knowledge you may use, and it is the only corpus you can reach.

TOOL

```
retrieve(query) -> up to 8 passages, each returned as
    [<identifier>] <question> / <accepted answer>
```

The query is natural language. Call it as often as the work needs.

WHEN YOU ARE GIVEN WORK

1. Retrieve from your corpus what the work needs.
2. Answer the work you were given, and nothing beyond it.
3. State the identifier of every passage your answer rests on, in the form [MS-XXXXXXXXXXXX]. The team can use only what arrives with its identifier.
4. If your corpus does not cover the work, say so and stop.

The answer format, appended to every question

Your final answer must state the identifier of every passage it rests on, in the form [MS-XXXXXXXXXXXX], exactly as the passage was returned.

The single agent

You are answering a user’s question on your own. You may search the question-and-answer corpora of 51 Stack Exchange communities, whose public profiles are listed below. These corpora are the only knowledge you may use.

QUESTION q, with the answer format every method receives>

PROFILES $\langle \text{site name, expertise scope and topic tags, 51 of them} \rangle$

TOOL

```
retrieve(site, query) -> up to 8 passages from that site’s corpus,
    each returned as [<identifier>] <question> / <accepted answer>
```

Name one site per call. Call it as often as the question needs.

Retrieve what the question needs, then write one answer.

The single-agent prompt carries no plan and no advice on how to use one finding to find the next. Every method brings its own mechanism for that, and the reference is the case with none, so whatever it does with a finding it does with one context and its backbone alone. It is told about identifiers only through the answer format, as a team’s final writer is.

The expert-discovery probe

Below is a user’s question and the public profiles of 51 expert agents.

QUESTION <q, as a method receives it>

PROFILES <site name, expertise scope and topic tags, 51 of them>

FINDING <the finding of the upstream unit, or NONE>

Name every community you would recruit to answer QUESTION, and for each one say which words you are going by.

Go by QUESTION and PROFILES, and by FINDING when it is given. Name a community only if something in front of you points to it. Do not guess from what such a question usually needs.

Reply with one line per community:

<site name> <- <the words you went by>

The probe runs twice on the same task. With FINDING set to NONE it must not name the downstream community, and with the upstream finding supplied it must. An edge that fails either half leaves its task in C2. The tag-overlap baseline of § B.1 runs alongside it and uses no model.

Construction reuses the agent prompt for everything else it needs. A unit is solved for local solvability (§ B.3) by giving its own expert agent the unit as its work, and a dependency is certified (§ B.2) by running the same agent twice on the downstream unit, once with the predecessor finding prepended to that work and once without. The subsumption test of § B.1 is the same call with another unit’s passages supplied in place of retrieval. All four prompts and the answer format ship with the code, together with the profiles and the corpora they read from, so every model call MAScope makes on its own behalf can be replayed.

I LIMITATIONS AND DATA

I.1 WHAT THE FINDINGS COVER

Every number we report on MAScope comes from one family of tasks, one backbone and one set of method configurations. The preliminary study of § E is the one exception, and it runs on two existing benchmarks for exactly that reason. The tasks are information-solving, where a contribution is a finding retrieved from a corpus and adoption means carrying that finding into an answer. The backbone is `claude-sonnet-5` throughout, for the twelve methods, the 51 agents, the construction-time solves and the preliminary runs alike. The configurations are each method’s published defaults (§ D.1). A capability rate read here therefore describes one method, at this scale of task, on this backbone, in its default shape.

Two things bound how a number here should be read. The five runs differ only in the sampling seed (§ C.1), so a spread we report says how much a rate moves between runs and says nothing about a different backbone. A stronger or weaker model would move every rate, and what we expect to hold under it is the ordering, since the ordering rests on what a mechanism does with a finding and the level of any single agent enters all twelve alike. Acceptance is settled by string against the surface variants recorded with each term (§ C.2), so a correct answer phrased outside them is scored as a miss. Expert selection compares agent sets and is untouched, while every rate downstream of a term match is lowered by this. Those rates are lower bounds on what each method achieved. The three mechanisms of § 4.3 are properties of what a mechanism does with a finding produced at run time, and that is the level at which we state them.

I.2 WHAT THE ENVIRONMENT LEAVES OUT

A contribution in MAScope is a piece of retrieved knowledge, and it is complete the moment an agent states it with its identifier. Collaboration also happens over things that change while the team works, and three kinds of it sit outside this environment. Agents write no code, so no contribution is a patch that a later agent has to build on. They share no tool state, so what one agent does to the world never constrains what another can do next. And nothing carries across queries by default, so a team never accumulates a way of working, which § D.4 adds as an option and measures separately. Each of these gives a contribution a lifetime longer than one message, and following one would need a reference that tracks state as well as text. Extending the same tracing to them is the next step.

I.3 DATA AND RELEASE

The corpora derive from public Stack Exchange posts under CC BY-SA. We redistribute them under the same license, keep the attribution it requires, and strip user names, profile links and every other author-identifying field before a corpus enters the environment (§ A.3). The release ships the 51 corpora with their identifiers, the 2,766 queries with their references, family labels and preregistered dependency graphs, the twelve adapters, the evaluator, and the trajectories of all five runs with the stage label of every dependency, which is what lets a reader recount any rate of Table 2 on a population of their choosing.

Releasing the corpora publishes the identifiers with them, which bounds the memorization argument of § A.3. The argument holds for any backbone trained before the release, since the identifiers did not exist earlier. A backbone trained afterwards could have read them, and that is why we keep the minting key. With the key the evaluator re-mints the identifier set of any released task, so a later model can be scored against identifiers it cannot have seen, on the same corpora and the same references. The code is released under the MIT license and the derived corpora under CC BY-SA, both at the anonymized repository linked from the abstract.